



Generative AI Academy

Function calling and Agents

Query BFSI customer information

This web API presents an interface to ask questions on customer information stored in a database.

user_input

Do customers who default maintain a low balance?

Clear

Submit

output

The average balance for customers who do not default is approximately 1552.84, while the average balance for customers who default is approximately -61.80.

Customers who default have a significantly lower average balance compared to customers who do not default.

What is an Agent?

Definition

An **AI agent** is an **autonomous system** that

- uses a LLM as a reasoning and decision-making core
- combined with memory, tools, and an execution loop

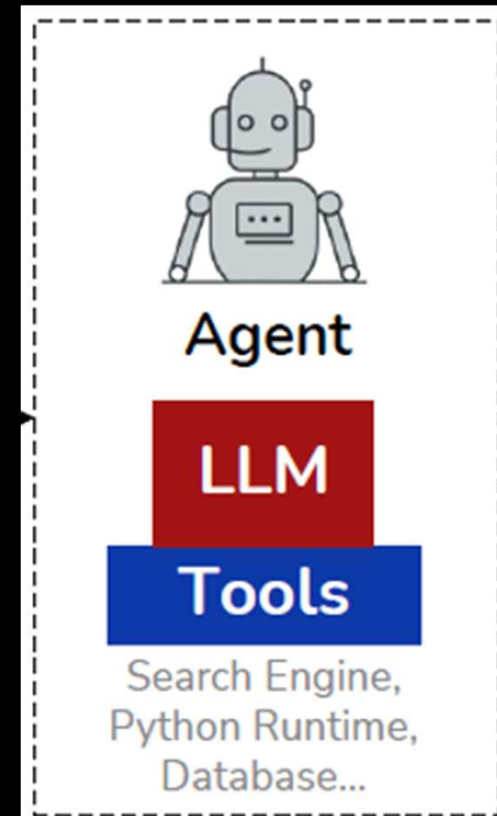
to perceive inputs, plan actions, and produce outputs aligned to a goal.

Perceive => parse input + context with the LLM.

Act => call tools/functions or generate text.

Achieve goals => follow user intent or multi-step objectives.

It is a **conceptual and architectural abstraction** to describe a certain *pattern of behavior* in applications built on top of LLMs.



How it maps to real world?

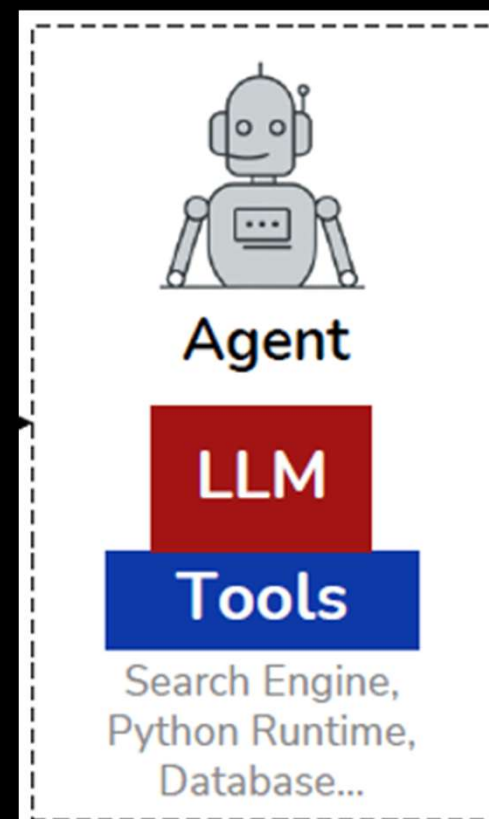
LLM → the reasoning core (text-in, text-out).

Tools/Functions → the external capabilities (APIs, databases, calculators, RAG pipelines).

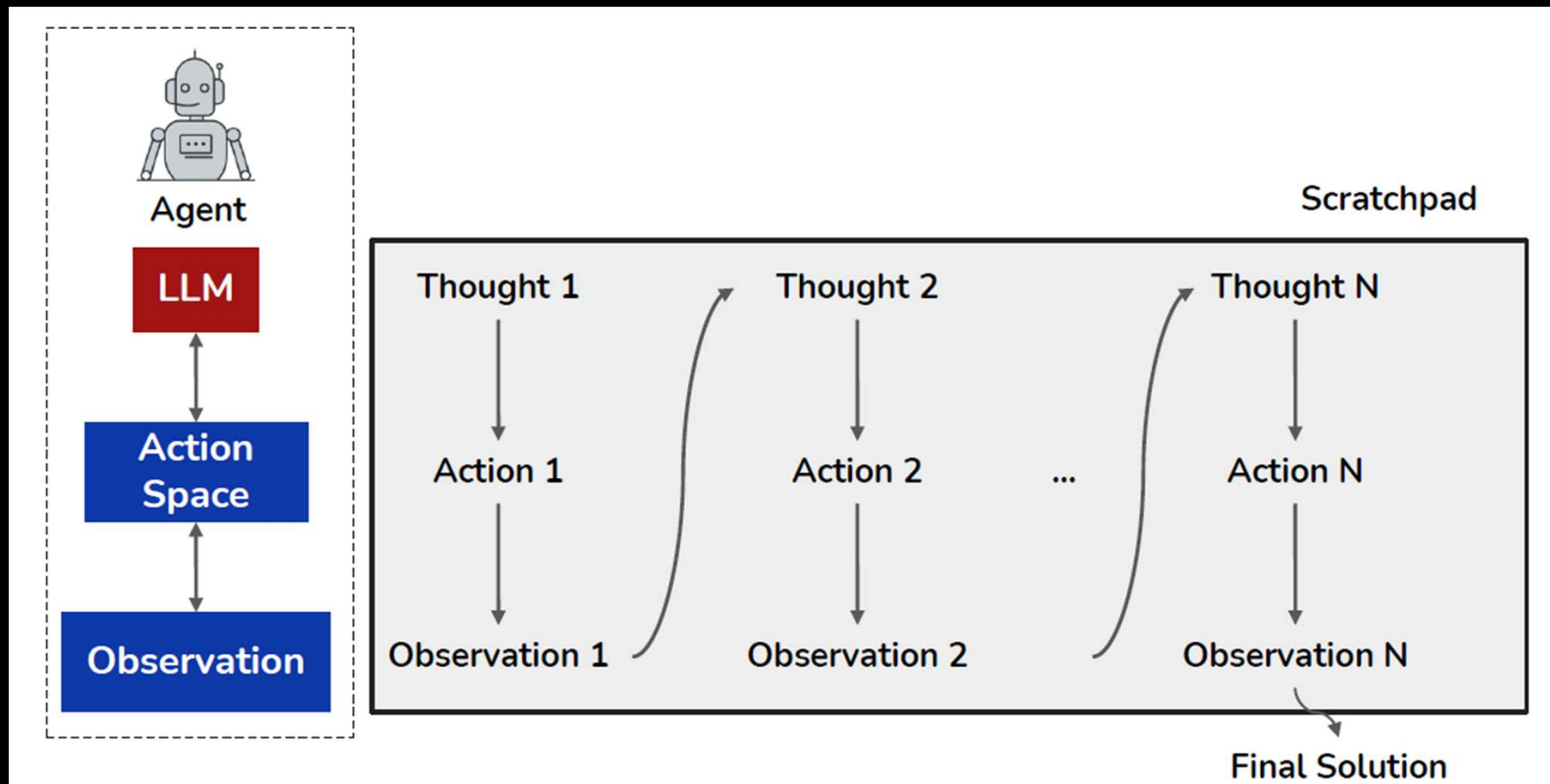
Memory → persistence mechanism (conversation state, vector DB, key-value store).

Control Loop → orchestration logic (can be as simple as ReAct prompting, or as complex as CrewAI, LangGraph, AutoGen, etc.).

Put together, **the whole application that stitches these pieces together is what we call an “agent.”**



ReAct paradigm (Reasoning and Action)



Example of an effective ReAct prompt

You are a reasoning and acting agent.

Rules:

- Always think step by step before acting
- Use tools only when necessary
- Do not hallucinate tool outputs
- After receiving an observation, reassess your plan

Available format:

Thought:

Action:

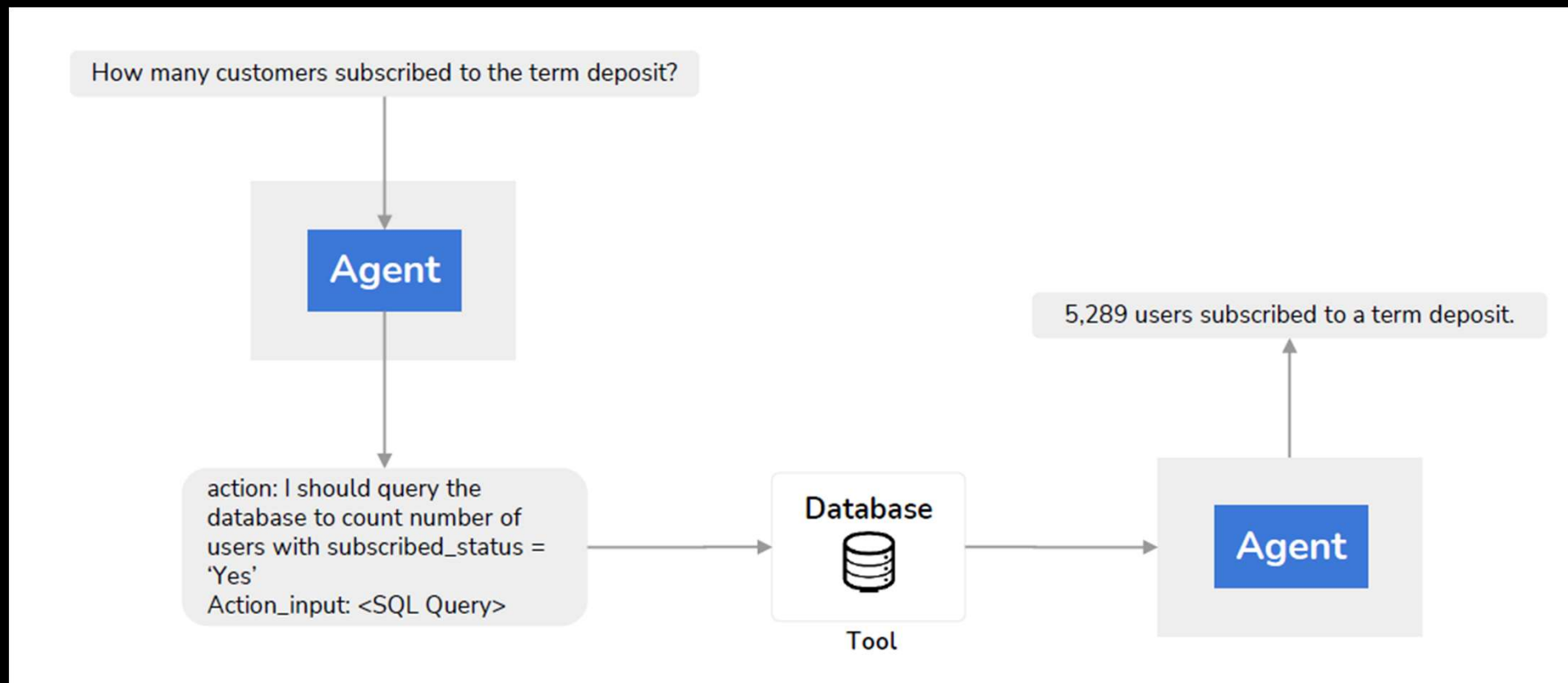
Action Input:

Observation:

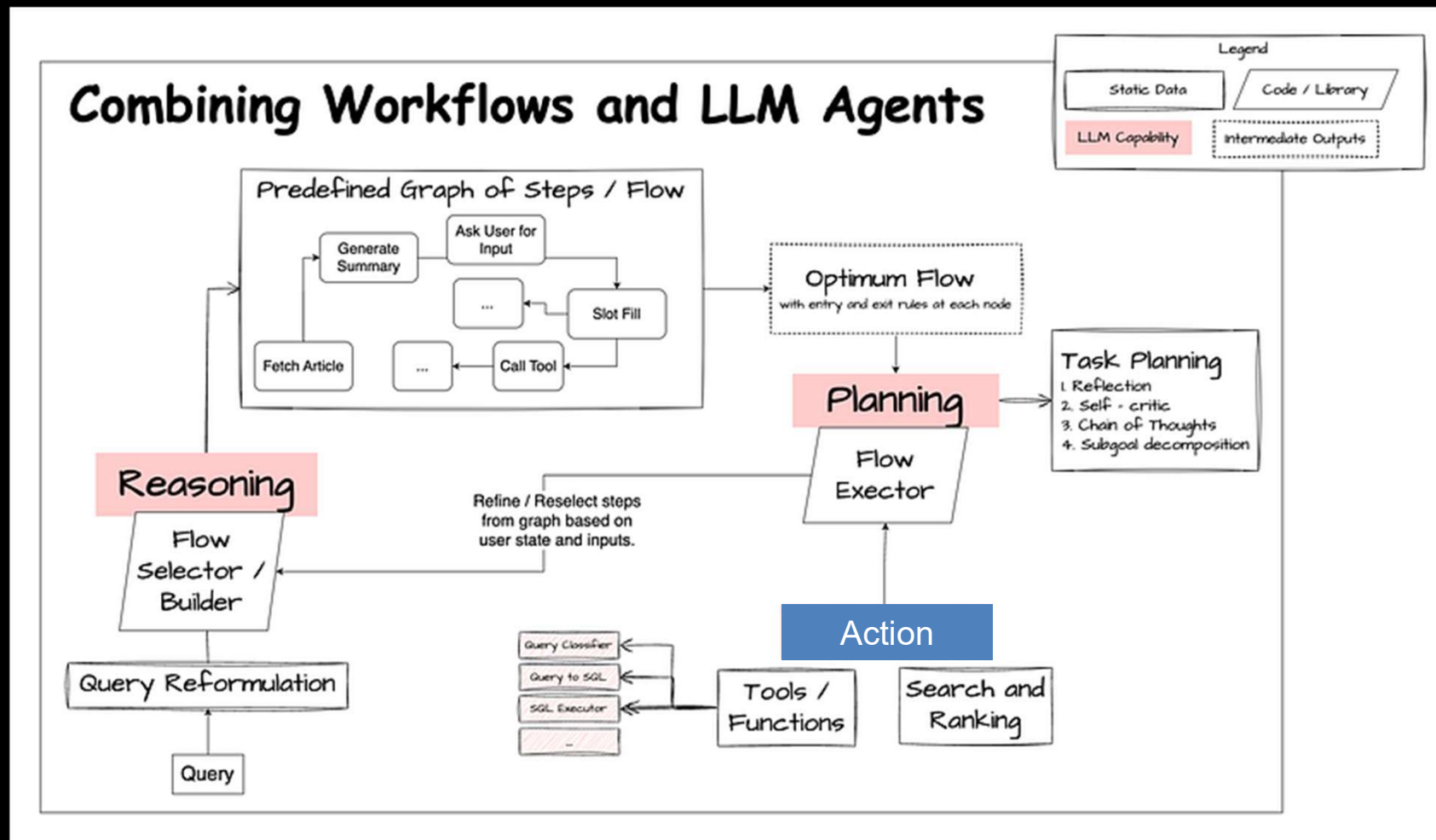
...

Final Answer:

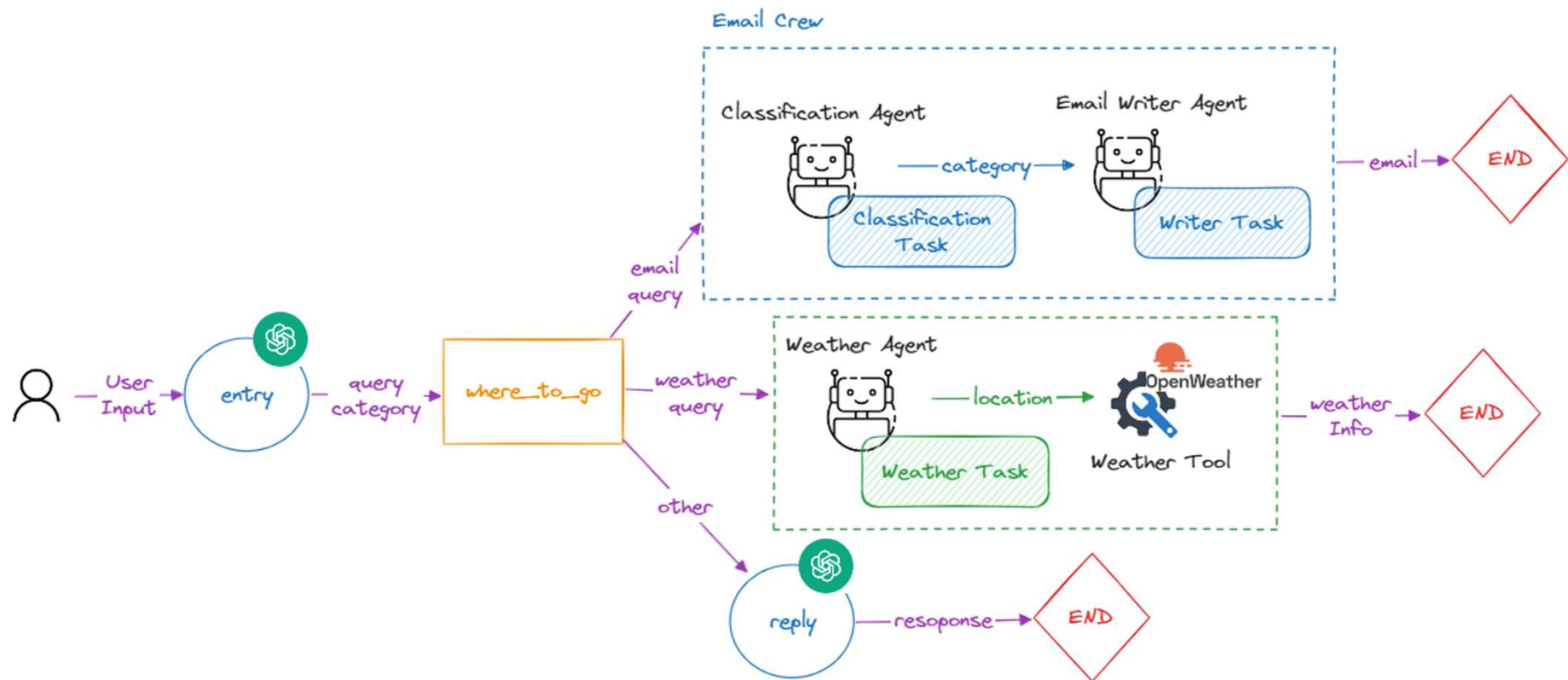
Agent workflow



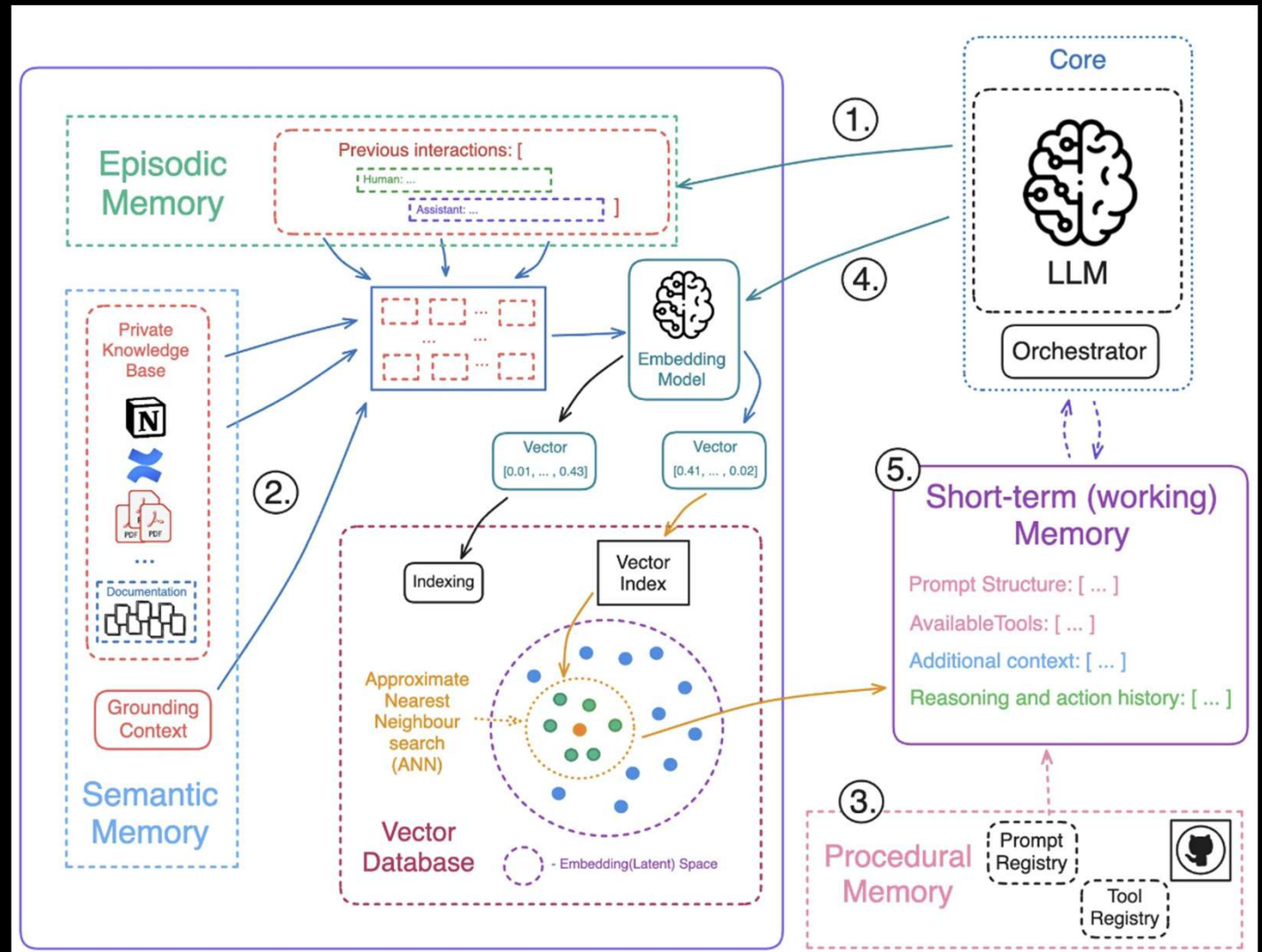
Workflow with Agents – Agentic AI



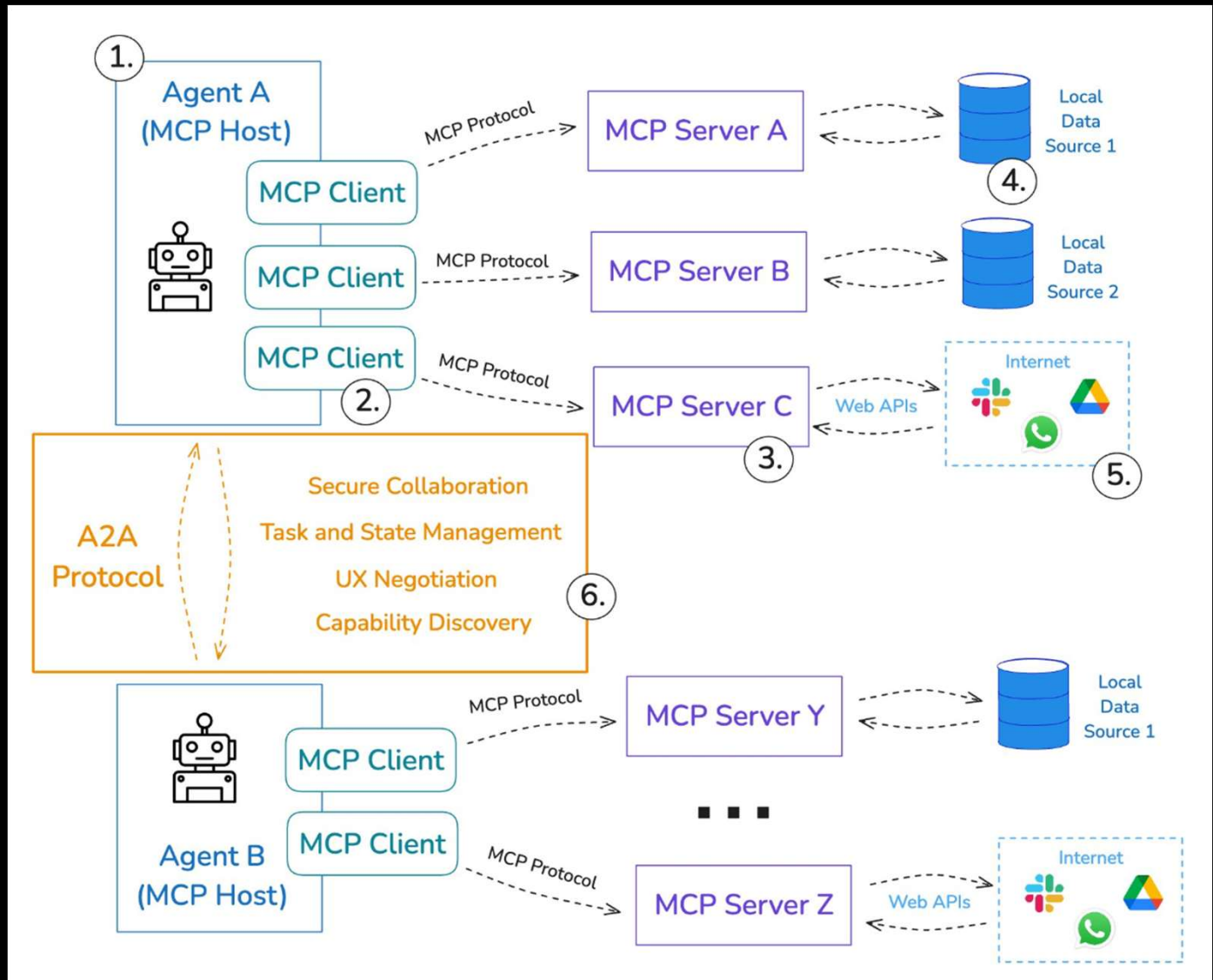
Workflow example



Agentic AI – Memory and State



Agent 2 Agent A2A



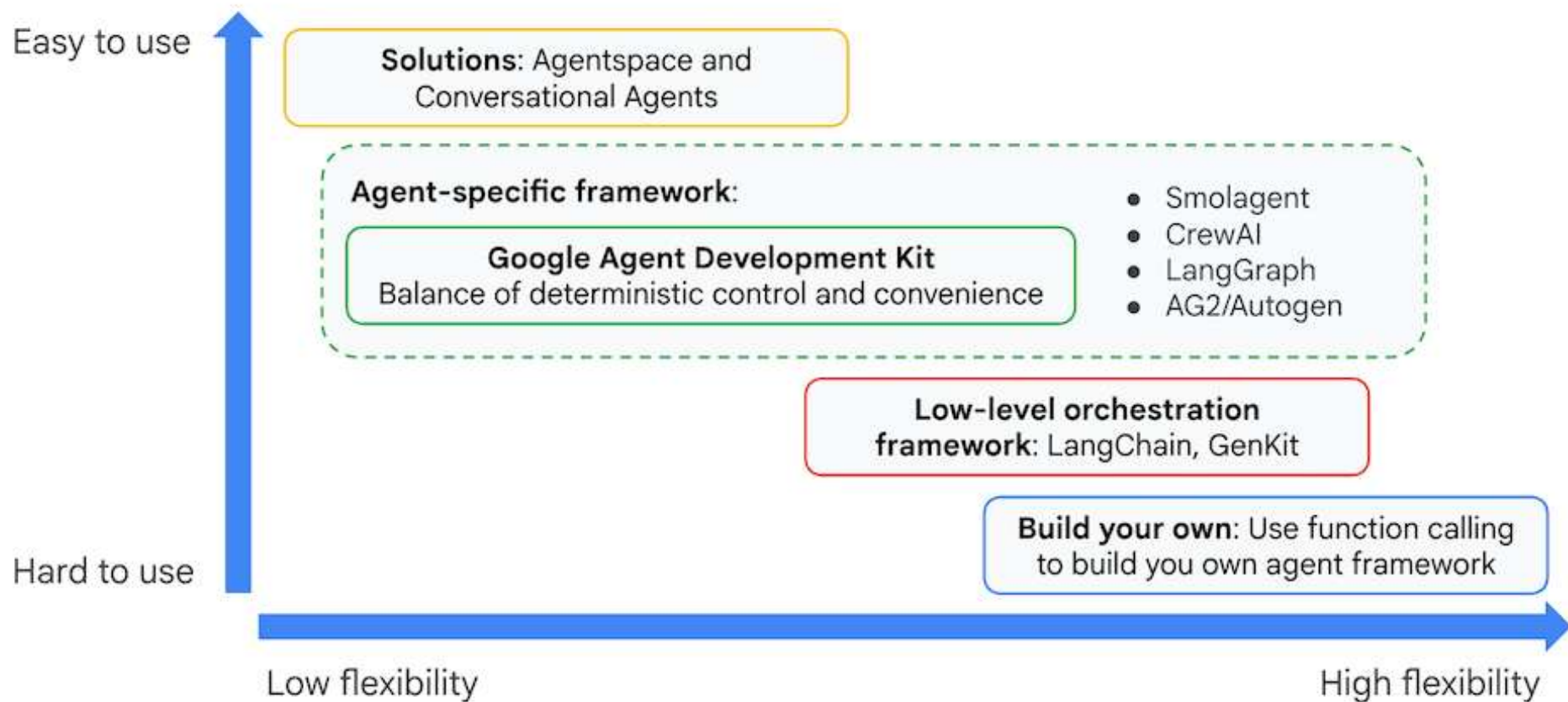
Function calling and Agents
Demos and hands-on

Frameworks to build Agentic Apps

Why are frameworks needed?

- Simplify workflow orchestration of multi agent flows
- Inter agent communication
- Ability for Agents to have a **state**
- Memory
 - Short term
 - Long term
 - Episodic (conversation boundaries / topic boundaries / task boundaries)

Agentic AI Development - Options



Popular Agentic Frameworks

Code Based

- Langchain Agents and Langgraph
- LLamaIndex Tools
- Open AI Swarm
- Crew AI
- And many more (RASA, AutoGen, SmolAgents etc.)

Low code/No code:

- n8n
- Flowise
- Amazon Bedrock Flows (<https://aws.amazon.com/bedrock/flows/>)
- Azure Agent Builder (<https://learn.microsoft.com/en-us/power-apps/maker/canvas-apps/agent-builder>)

Considerations

- Langgraph, Crew, Swarm offer powerful orchestration and workflow components
- Application design considerations will involve properly marrying these capabilities with LLM capabilities around summarization, generation, reasoning and planning
- Rule based workflows are perhaps better captured by hard wiring the flow using these tools for predictability and reliability

Crew AI

Primary Functionality

CrewAI is built around the concept of a "crew" of agents working together. It uses a **structured process flow** to execute tasks sequentially or in parallel

Customization

Offers role-based specialization, where agents are defined by roles and goals. This framework abstracts complexity by providing high-level constructs for task management.

Workflow Management

Uses a **sequential execution model** by default, but can be scaled by running multiple crews in parallel.

Ease of Use

Known for excellent documentation and a robust community, making it easier for users to get started

LangGraph

LangGraph models agentic workflows as a state machine (a graph), allowing for explicit control over loops, edges, and state transitions.

LangGraph's fundamental architecture is a **state graph**, making it ideal for processes that are not strictly sequential and require loops (cycles).

While other frameworks can simulate this, LangGraph provides a more natural and robust way to define and manage such complex, state-dependent flows.

LangGraph uses typed state objects that are passed between nodes and updated immutably. This provides structured, type-safe state management that persists across the entire graph execution, making it ideal for complex multi-step workflows.

It also provides a visual tool, LangGraph Studio to build such graphs visually

Microsoft AutoGen

- Opensource library for building Agentic Applications - Microsoft is primary contributor
- Multi-agent workflows with asynchronous, event-driven architecture
- Support for both Python and .NET programming languages
- Customizable and conversable agents which integrate LLMs, tools, and humans
- AutoGen Studio - a low-code interface for building multi-agent workflows

A2A (Agent to Agent)

- A2A is a communication protocol, not a framework or library.
- Released by Google in Apr 2025, open-sourced and released to Linux Foundation in Jun 2025
- Designed so agents (regardless of their internal codebase or host framework) can send secure, interpretable requests and responses to one another—for example, over HTTP using JSON-RPC 2.0.
- Enables agents built in different systems (AutoGen, LangGraph, proprietary) to collaborate without manual integration.
- Handles authentication, authorization, and streaming updates; acts as a “universal translator” for agents.

Pydantic

Pydantic is a Python library for data validation and parsing.

It uses standard Python type hints (like `str`, `int`, `bool`) to define how your data should look.

- When you pass data (like a JSON object or a dictionary) into a Pydantic model, it:
- Validates that all required fields are present.
- Coerces data to the correct types (e.g., it can turn the string "123" into the integer 123).
- Raises clear, specific errors if the data is invalid.

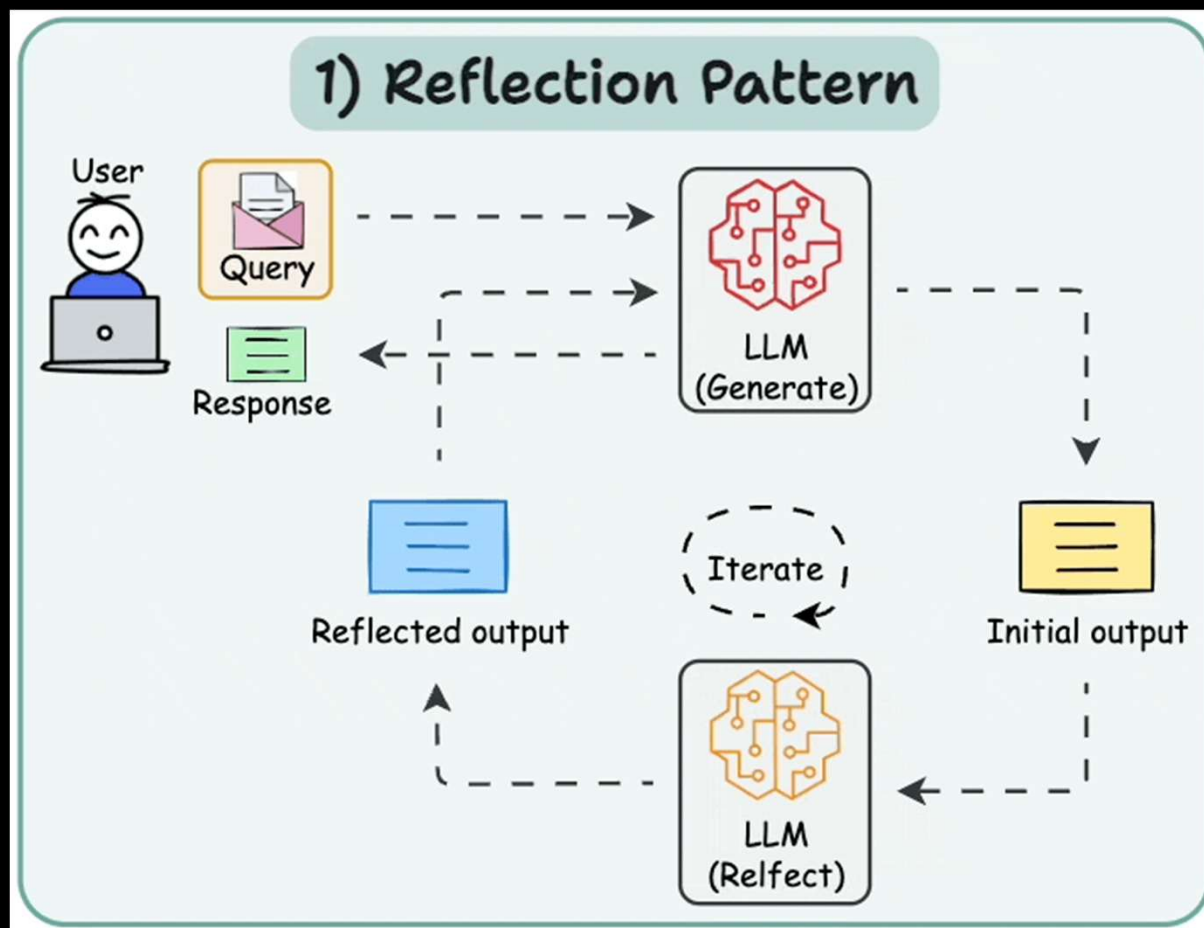
Think of it as a strict, enforceable data "blueprint" or "contract" for your functions and classes.

Pydantic is the "glue" that connects the unpredictable, text-based world of LLMs to the structured, reliable world of application code.

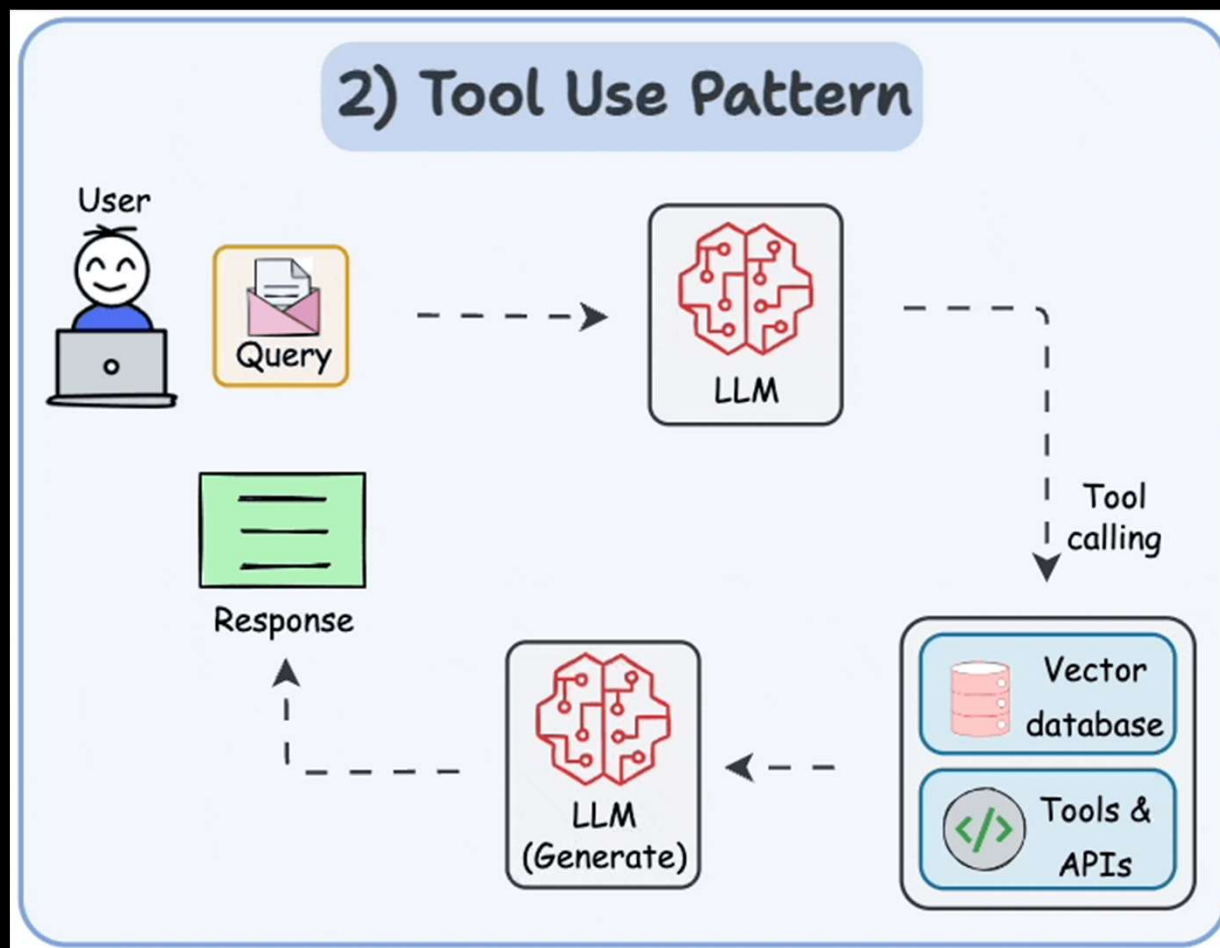
Demo of sample applications with LangGraph, Crew AI

Agentic AI Design Patterns

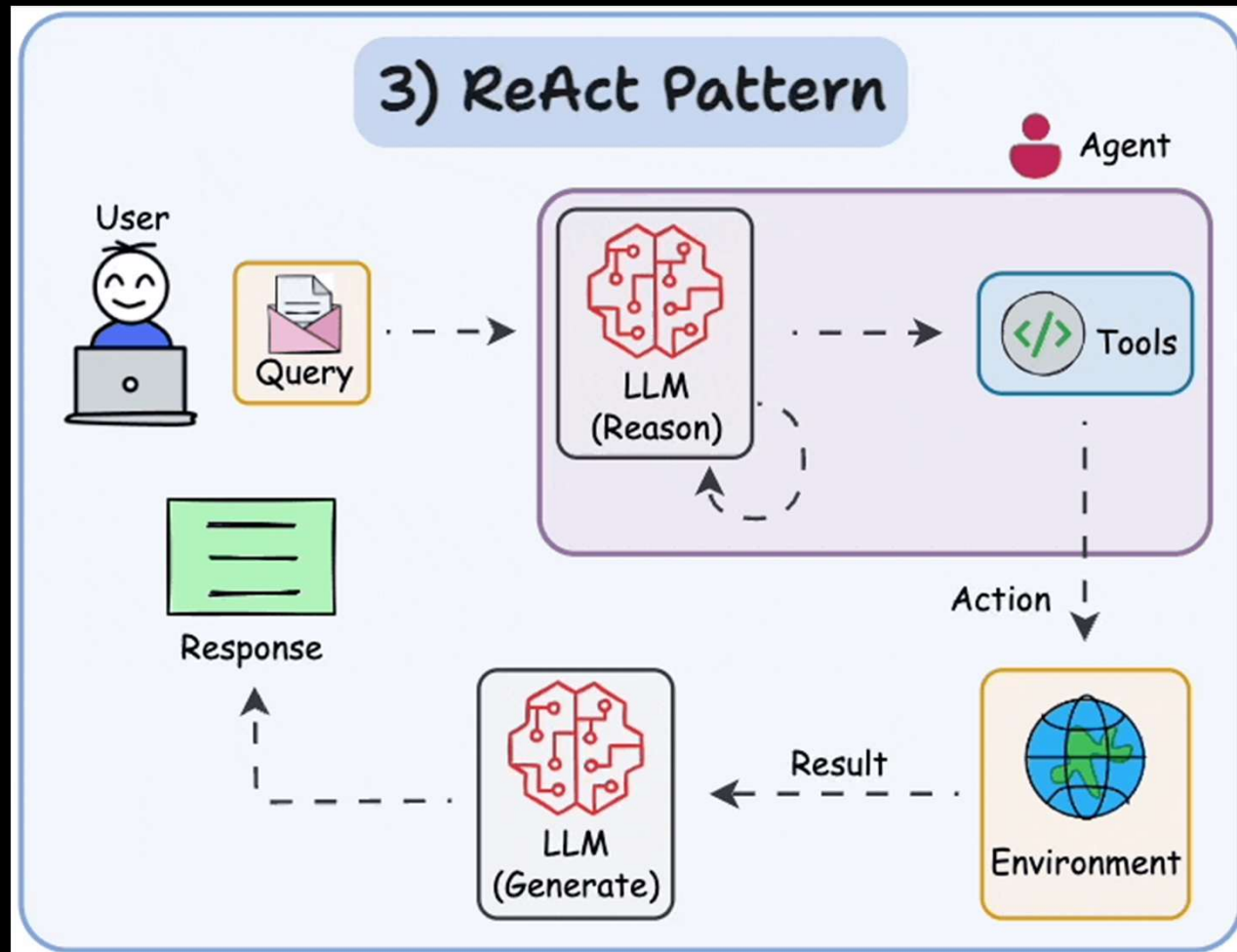
Reflection



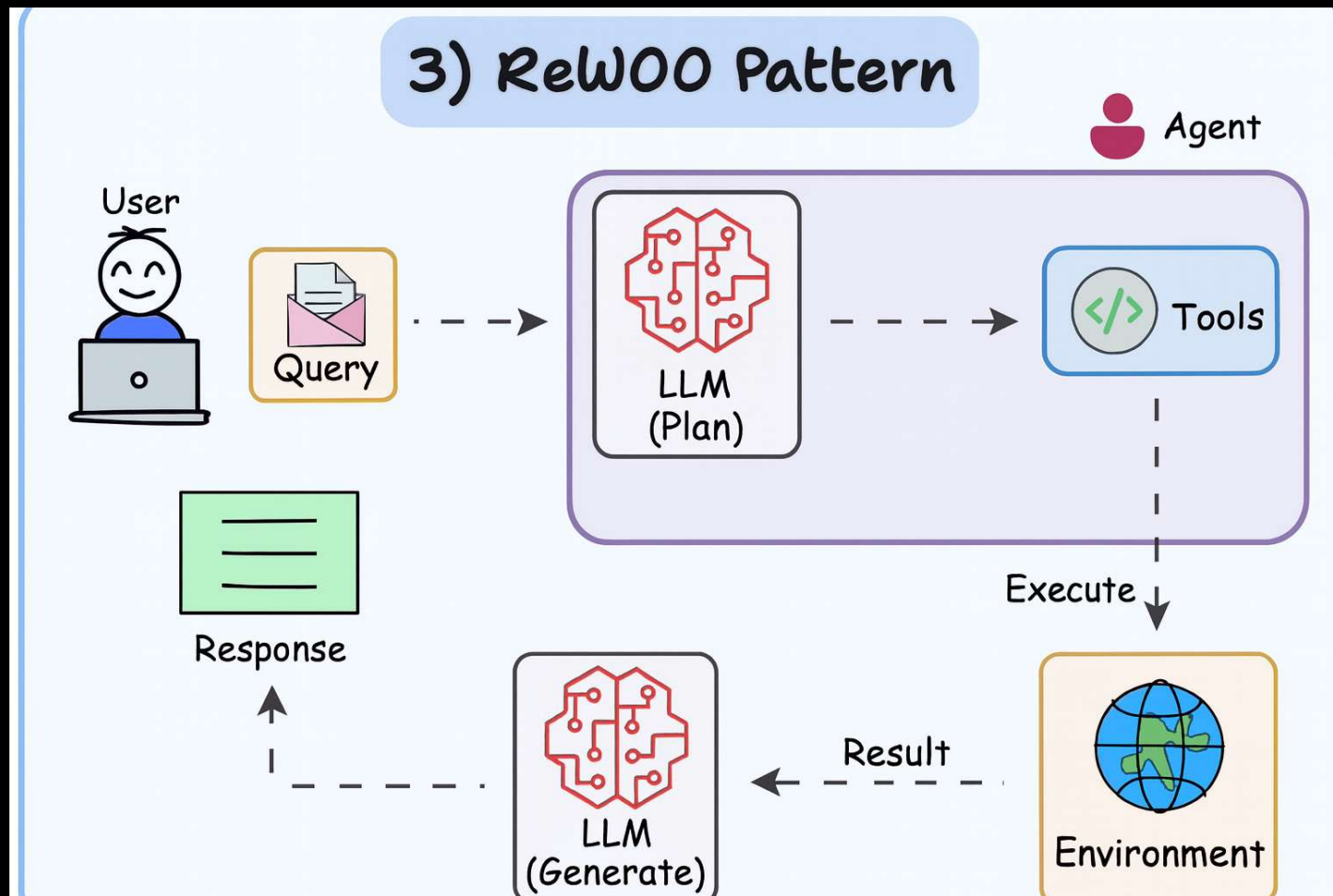
Tool Use



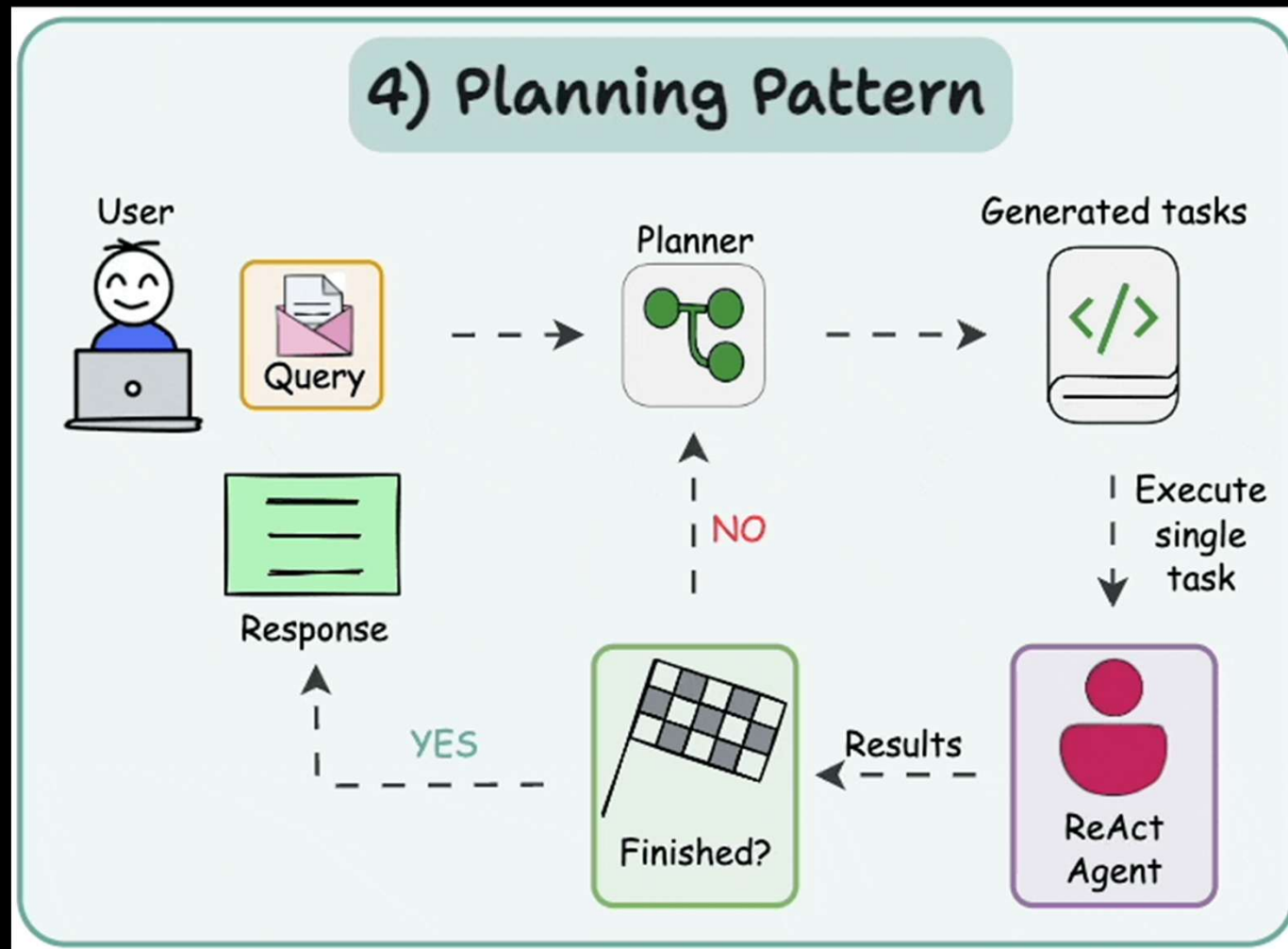
ReAct



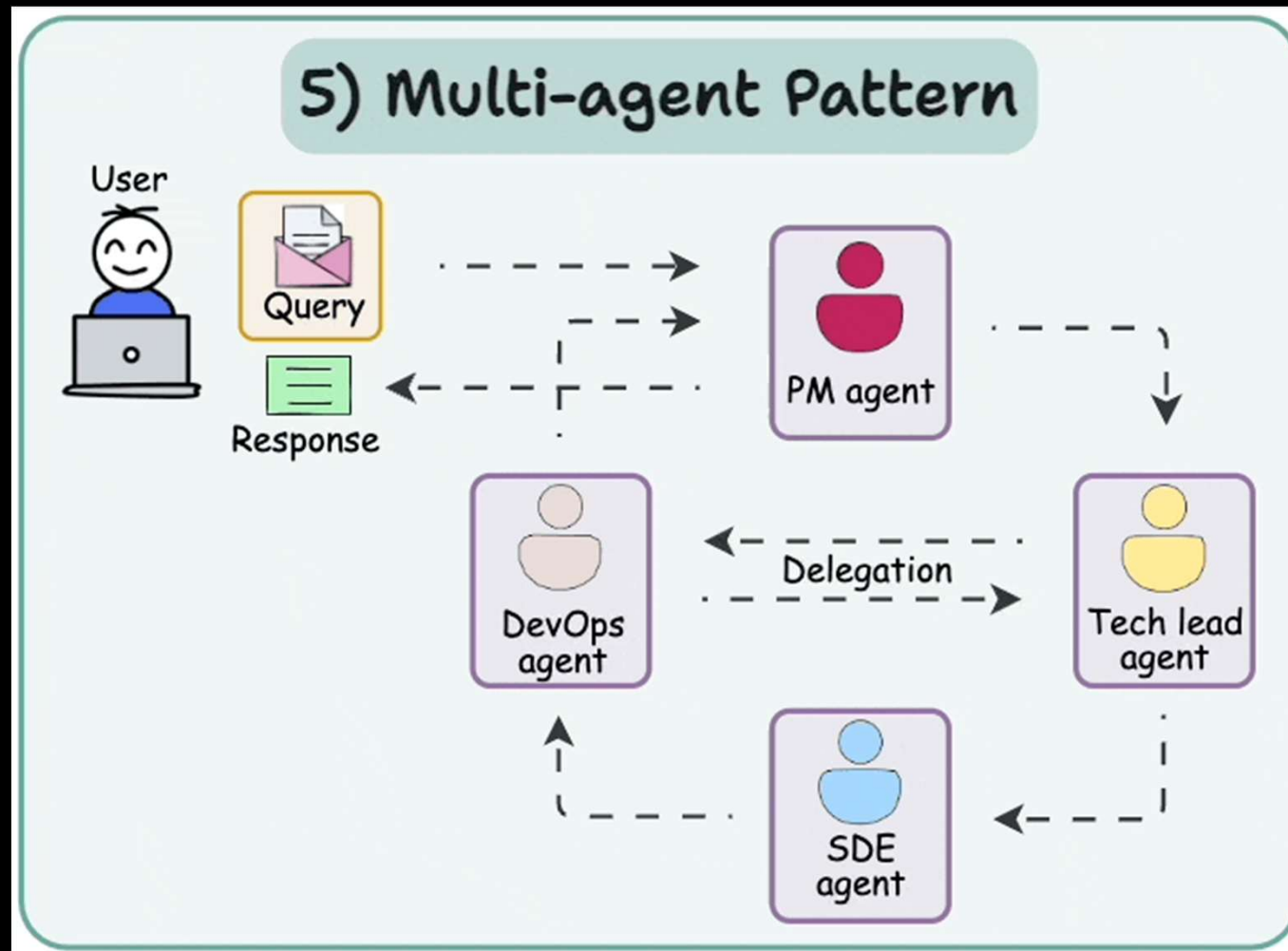
ReWoo (Reasoning Without Observation)



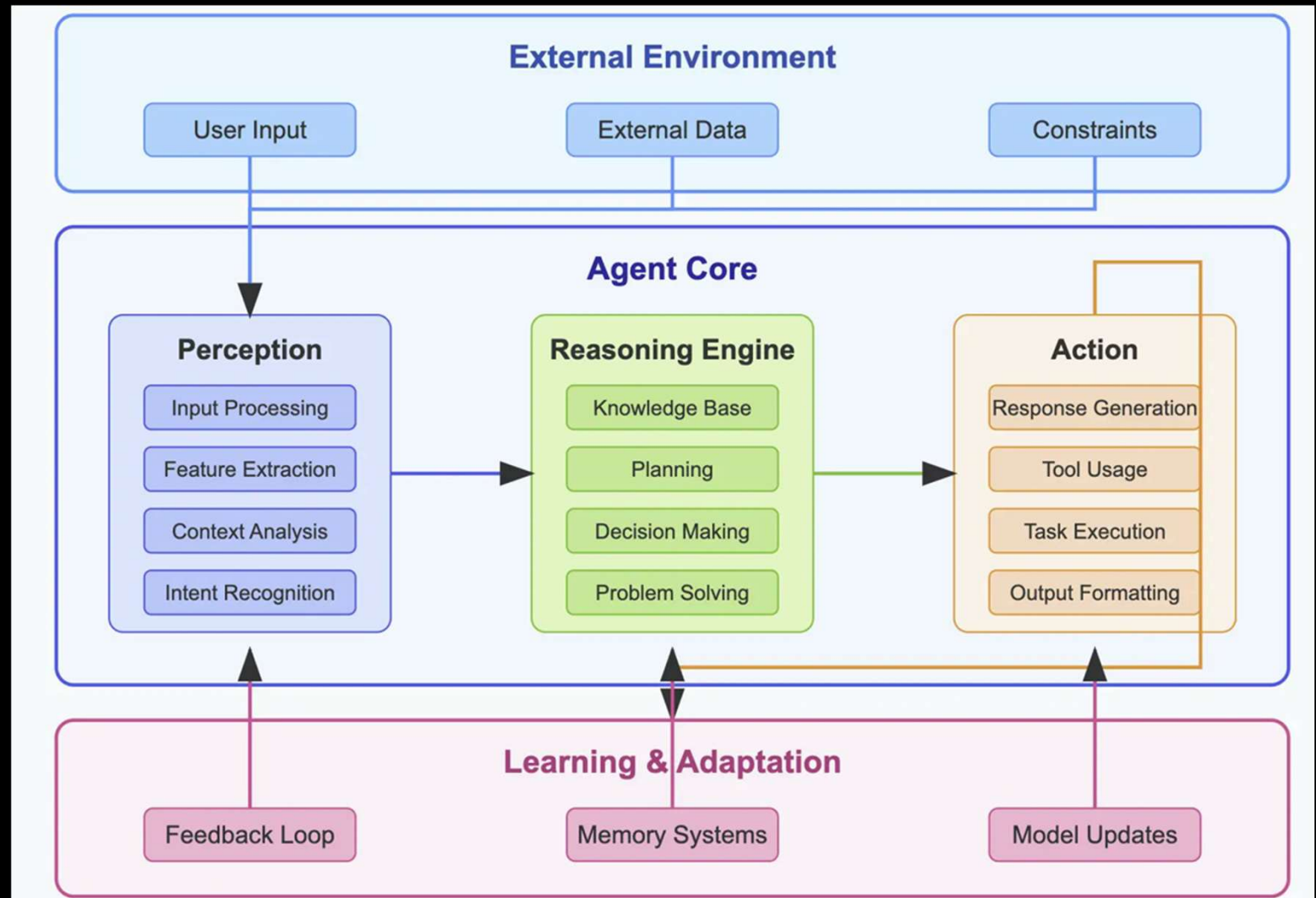
Planning Pattern



Multi agent – collaborative with memory and state



Multi agent architecture components



Ref: Salesforce AI Research [GitHub - SalesforceAIResearch/enterprise-deep-research](https://github.com/SalesforceAIResearch/enterprise-deep-research): Salesforce Enterprise Deep Research

AI Agents Stack

NOVEMBER 2024

VERTICAL AGENTS

 Decagon  SIERRA  replit  perplexity Harvey  MultiOn
 Cognition  FACTORY  All Hands  Dosu  Lindy  11x

AGENT HOSTING & SERVING

 Letta  LangGraph
 Assistants API  Agents API
 Amazon Bedrock Agents  LiveKit Agents

OBSERVABILITY

 LangSmith  arize
 weave  Langfuse
 AgentOps.ai  braintrust

AGENT FRAMEWORKS

 Letta  LangGraph  AutoGen  LlamaIndex  crewai
 DSPy  phidata  Semantic Kernel  AUTOGPT

MEMORY

 MemGPT  zep
 LangMem  mem0

TOOL LIBRARIES

 composio
 Browserbase  exa

SANDBOXES

 E2B
 Modal

MODEL SERVING

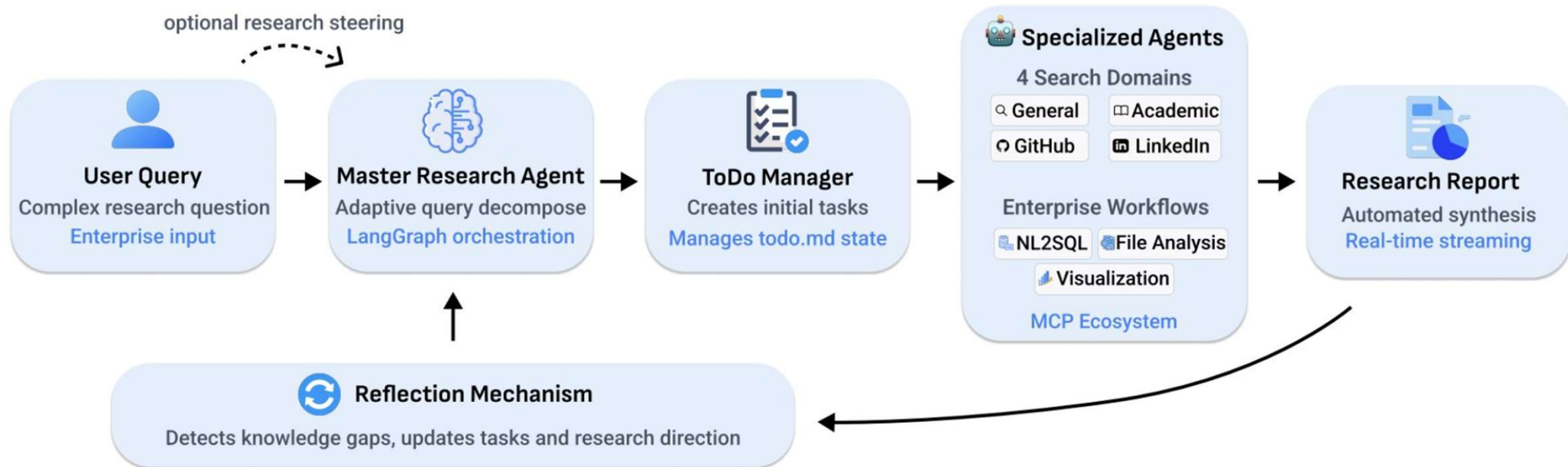
 LLM  ollama  LM Studio  SGL
together.ai  Fireworks AI  groq
 OpenAI  ANTHROPIC  MISTRAL AI  Gemini

STORAGE

 Chroma  drant  milvus
 Pinecone  Weaviate
 NEON  supabase

Example: Multi agent architecture for Enterprise Research

EDR Pipeline: Multi-Agent Architecture for Enterprise Research



Ref: Salesforce AI Research [GitHub - SalesforceAIResearch/enterprise-deep-research](https://github.com/SalesforceAIResearch/enterprise-deep-research): Salesforce Enterprise Deep Research

The diagram illustrates the MLOps lifecycle for GenAI, consisting of 11 numbered steps:

- Problem**: Initial problem identification.
- Build Prototype**: Initial model development.
- Define Performance Metrics**: Establishing evaluation criteria.
- Define Evals**: Setting up evaluation infrastructure.
- Build PoC**: Building a Proof of Concept.
- Expose to Users**: Initial deployment.
- Tracing**: Monitoring model performance.
- Human Feedback**: Incorporating user input.
- Observability Platform**: Central monitoring hub.
- Run Evals on Traces**: Evaluating model performance on real data.
- Failing Evals**: Identifying areas for improvement.
- Prompt Engineer / Finetune / Improve Topology**: Iterative model improvement.
- Build Application (v.X)**: Final application development.
- Production Monitoring**: Continuous monitoring in production.
- Alert**: Triggering alerts for issues.

The diagram also includes a legend for the GenAI Model (API, Managed, On-Prem) and a list of components used in the lifecycle, such as Guardrails, RAG, Data Preparation, Prompt Engineering, Agents, Tools, Model Routing, Reranking, and Topology.



Agent

The core worker that reasons, plans, and collaborates to complete tasks.



Tools

Give agents extra powers to access APIs, run code, or search data.



Session Services

Keep track of conversation context, history, and memory.



Callbacks

Add custom checks, logging, or behavior tweaks at key steps.



Artifacts

Manage files or data (like PDFs or images) linked to a session.



Runner

The execution engine that coordinates agent actions and workflows.



Demos of Agentic AI Design Patterns

MCP: Model Context Protocol

Models are only as good
as the **context** provided to them.
— Anthropic

M x N Problem

The complexity of point-to-point connections



The M x N Problem

⚠ 4 applications × 2 services = **8 connections**

</> Each connection requires separate SDKs, authentication, error handling

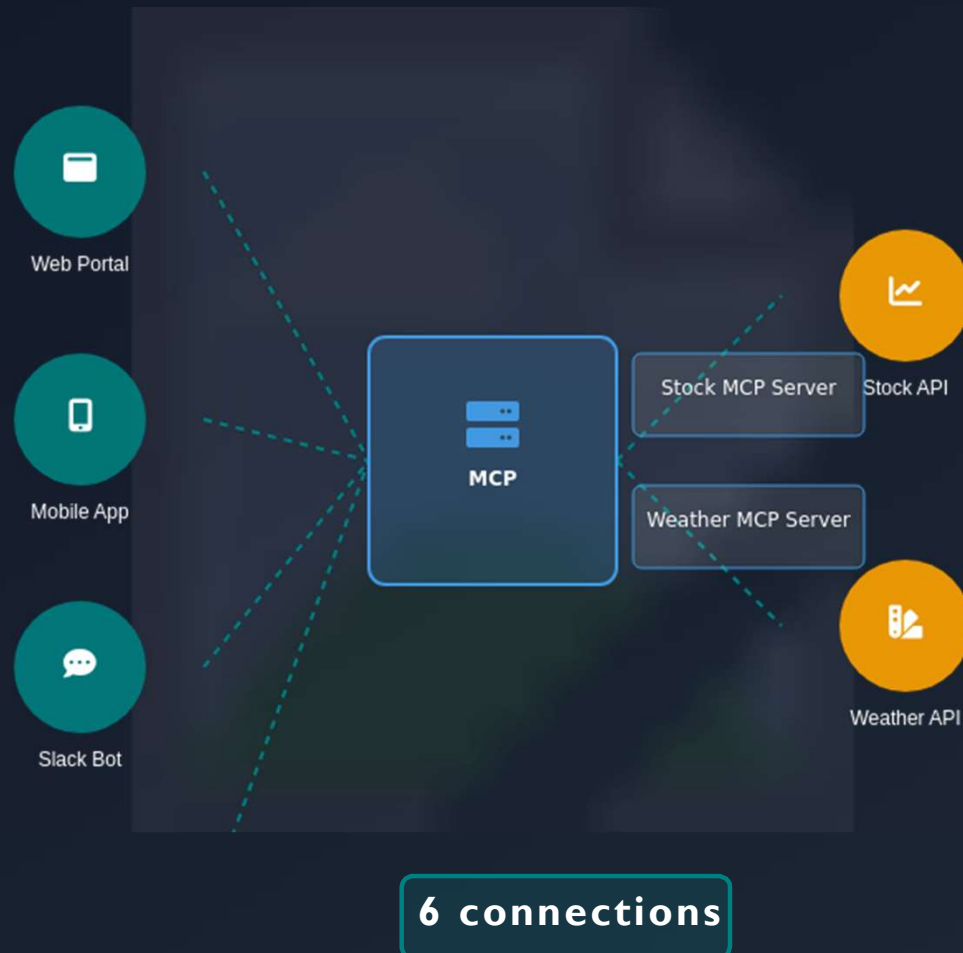
✂ Different authentication flows: API keys, OAuth, etc.

🔧 Custom response mapping for each service

Each new application or service **exponentially increases** integration

The MCP Solution: Standardized Integration

Transforming $M \times N$ to $M+N$ with standardized clients and servers



The M+N Solution

- ✓ 4 applications + 2 services = **6 connections**
- 🔌 Each app uses a standard MCP client interface
- 📦 Each service is wrapped in a standardized MCP server
- 🔄 Uniform protocol for all connections

*New applications or services only require **one new connection** to the MCP ecosystem*

What is MCP?

The **Model Context Protocol** standardizes how **AI applications** and **agents** interact with external **tools** and **data sources**.

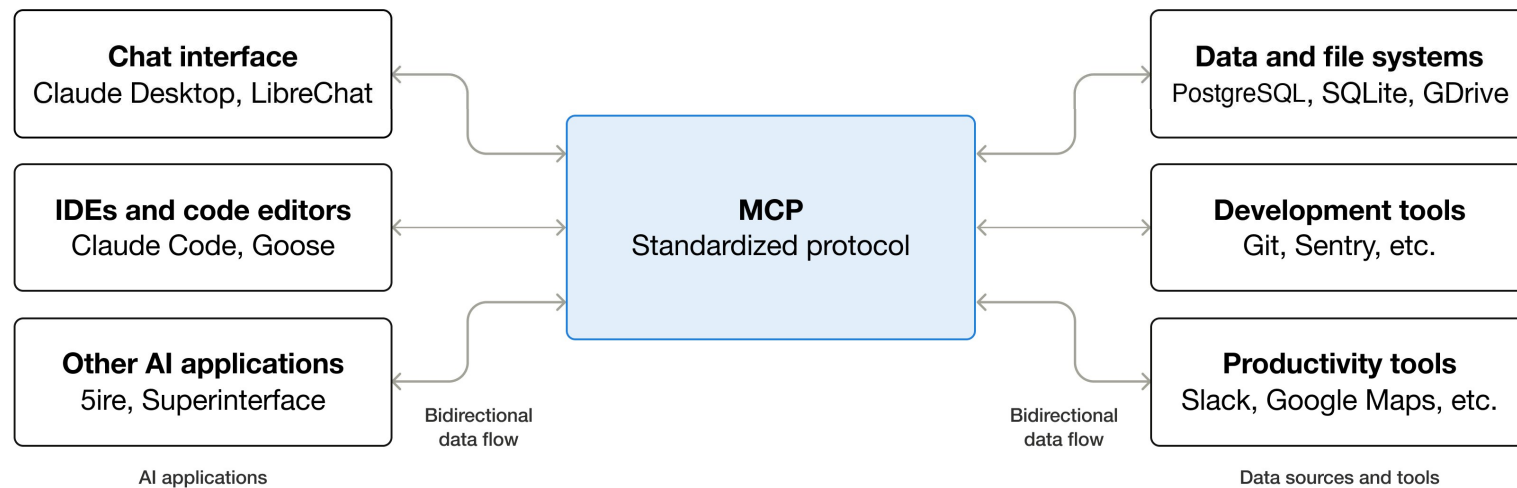
MCP is a **low-level application protocol** (the “USB-C of AI apps”) that standardizes connectivity between any App/Agent/Tool and any external tool or data source.

It's a **specification** you build against, with SDKs and server/client reference implementations

MCP – Standardized the interface

MCP acts like a “USB port”, providing AI applications with a **universal interface** to access **relevant context** and **perform actions** on other systems.

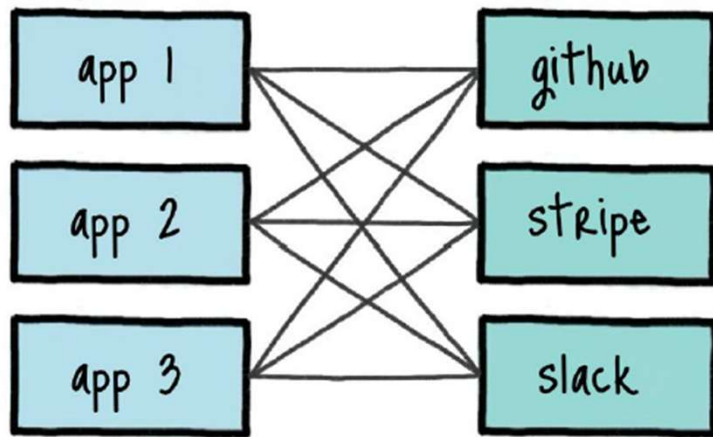
MCP – Standardized the interface



<https://modelcontextprotocol.io/>

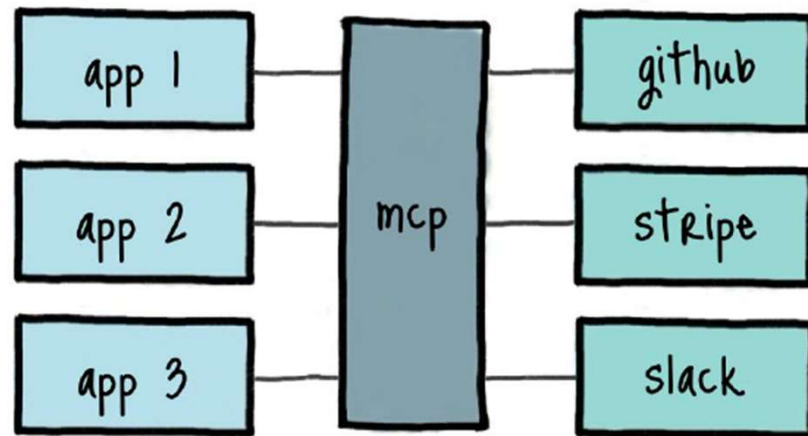
From fragmented to standardized

before mcp - fragmented development
 $m \times n$ (9 custom integrations)



each application requires a custom integration with each tool

after mcp - standardized development
 $m + n$ (6 connections)



once your application is mcp-compatible, you can connect to any server with zero additional work

MCP – server side simplicity

Any team or vendor can spin up an MCP server that exposes one or more capabilities

- GitHub-connector
- Postgres-connector
- shell-executor, etc.

The servers speak the same JSON-RPC wire protocol, so any MCP-aware client can plug in.

MCP – client side simplicity

From the client's perspective

- Just point your MCP SDK at the server's URL (and credentials) and immediately gain all its functions.
- No per-tool SDKs
- No custom wiring
- Enables discovery

Real World scenario – Finance portfolio Q&A App

Scenario:

Your portfolio-management team in a private-bank wants an on-demand assistant that can answer natural-language questions about client portfolios, flag any trades that may breach the bank's risk limits, and generate quick compliance memos.

How MCP helps?

- Data Connectors
 - Postgres/snowflake/sharepoint/S3
- Uniform client SDK
 - Agents use the MCP client APIs to discover and access the services in a uniform manner

Is MCP Server offering same functionality as API Gateways?

Traditional API Gateway / Backend	MCP Server Layer
✓ Full control over routing, auth, rate-limits, SLAs	✓ Leverages that control: proxies through your gateway, reusing the same OAuth/JWT flows
✓ Battle-tested middleware (logging, metrics, retries)	✓ Inherits all of the above—MCP sits on top of your existing services
✗ Requires custom client libraries or code-gen per service	✓ Provides a one-size-fits-all JSON-RPC client schema consumable by any MCP-aware agent
✗ No native “tool discovery” for LLMs	✓ Agents can introspect available methods, parameters and docs at runtime

MCP specification: communication protocol

It uses **JSON-RPC 2.0** as the transport and message format.

All requests and responses (e.g. tools/list, tools/call, resources/list) follow JSON-RPC semantics:

Request from MCP Client

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "tools/list",
  "params": {}
}
```

Response from MCP Server

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "tools": [
      {
        "name": "get_weather",
        "description": "Get current weather for a city",
        "inputSchema": { "type": "object", "properties": { "city": { "type": "string" } } }
      }
    ]
  }
}
```

Creating an MCP Server

Python and Javascript/Typescript are popular languages supported

Node.js and Python packages are available that enable creating our own MCP Servers.

Key parts:

- Metadata of the tools: name, description, parameters, parameter descriptions
- Implementation of the tools
- Implementation of standard interfaces such as tools/list, tools/call
- Authentication and Authorization mechanisms

Exposing MCP tools via web service/http

Streamable HTTP transport enables exposing these tools as an HTTP Endpoint

FastMCP is an example of such a server implementation

Web Services such as Zapier MCP provide an easy mechanism to expose various tools as an MCP Endpoint

<https://mcp.zapier.com>

Using the tools in a client application

No-code integration with any client application that supports MCP

- Any Client application that supports MCP can use these tools without any coding
- Popular AI chatbots such as Claude, ChatGPT and IDEs such as Cursor, VS Code support MCP
- When given an MCP Endpoint, they can discover the tools and add them to your chatbot

Programmatic consumption in applications

- Python and Javascript libraries available to simplify programmatic discovery and integration of tools into any application

MCP and LLM

- MCP is a tools standardization specification, and hence does not directly talk to the LLM
- Similarly, LLM is MCP Agnostic

Then what does it have to do with Agents/Agentic AI?

- MCP helps as the tools can now be discovered by the application and passed onto the LLM
- Similarly, the function calls requested by the LLM can directly be sent to MCP Server for execution
- Hence the Application code becomes minimalistic, clean and there is good separation of concerns

So in our get_weather example, the Application will not implement any get_weather, It will:

- a) Discover get_weather via MCP Server(s) (via tools/list)*
- b) Pass the metadata to the LLM as tools/functions*
- c) When requested by the LLM, invoke the tool in the MCP Server (via tools/call)*

MCP and Agentic frameworks such as crew.ai

Agentic AI frameworks such as crew.ai provide good integration and capabilities to use MCP

```
# MCP tools become regular CrewAI tools
with MCPServerAdapter(server_params=...) as mcp_adapter:
    weather_tools = mcp_adapter.get_tools() # MCP → CrewAI tools

    agent = Agent(
        role='Weather Expert',
        tools=weather_tools, # Agents see these as normal tools
        # Agent has no idea these come from MCP!
    )
```

Code Demos of MCP capabilities

Agentic AI low code / no code visual builders

n8n

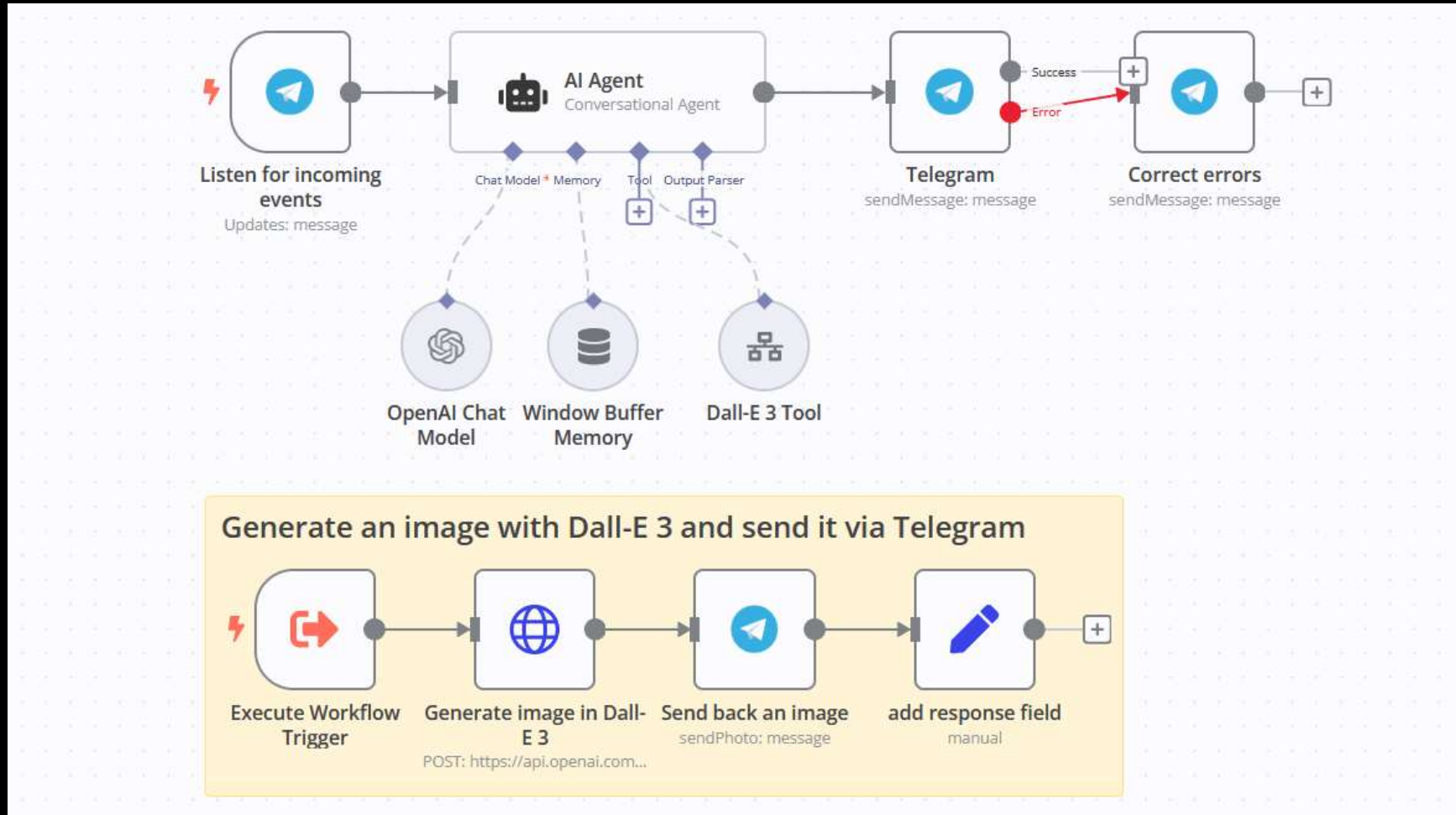
- n8n is an open-source, "fair-code" workflow automation platform that combines traditional workflow automation with advanced AI capabilities, specifically focusing on agentic AI.
- It allows users to visually design complex AI-powered workflows using a node-based interface, blending AI models, pre-defined logic, custom code, and human-in-the-loop interventions.

The Ultimate n8n Guide You'll Ever Need

<https://www.notion.so/The-Last-n8n-Guide-You-ll-Ever-Need-218071f74dc8802abec6ebee5c68a8db>

n8n example

Generate image with DALL-E 3 and send via telegram

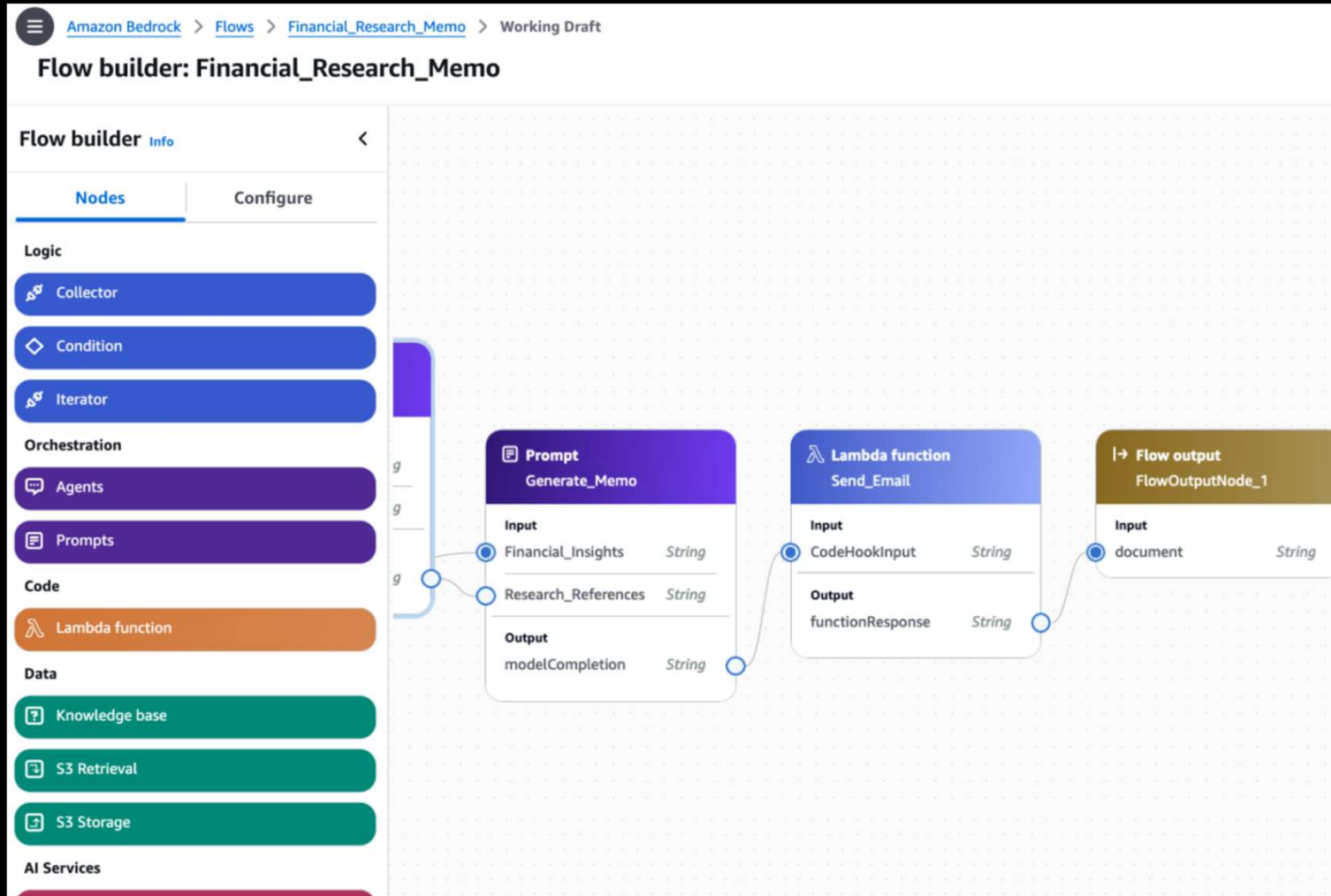


N8n – the company

- n8n is a company founded in 2019 by **Jan Oberhauser**.
- It operates with a "fair-code" distribution model, meaning its source code is visible and can be self-hosted, allowing for custom functions, logic, and apps.
- The company has secured significant funding rounds from investors including Sequoia, Felicis Ventures, and Highland Europe.

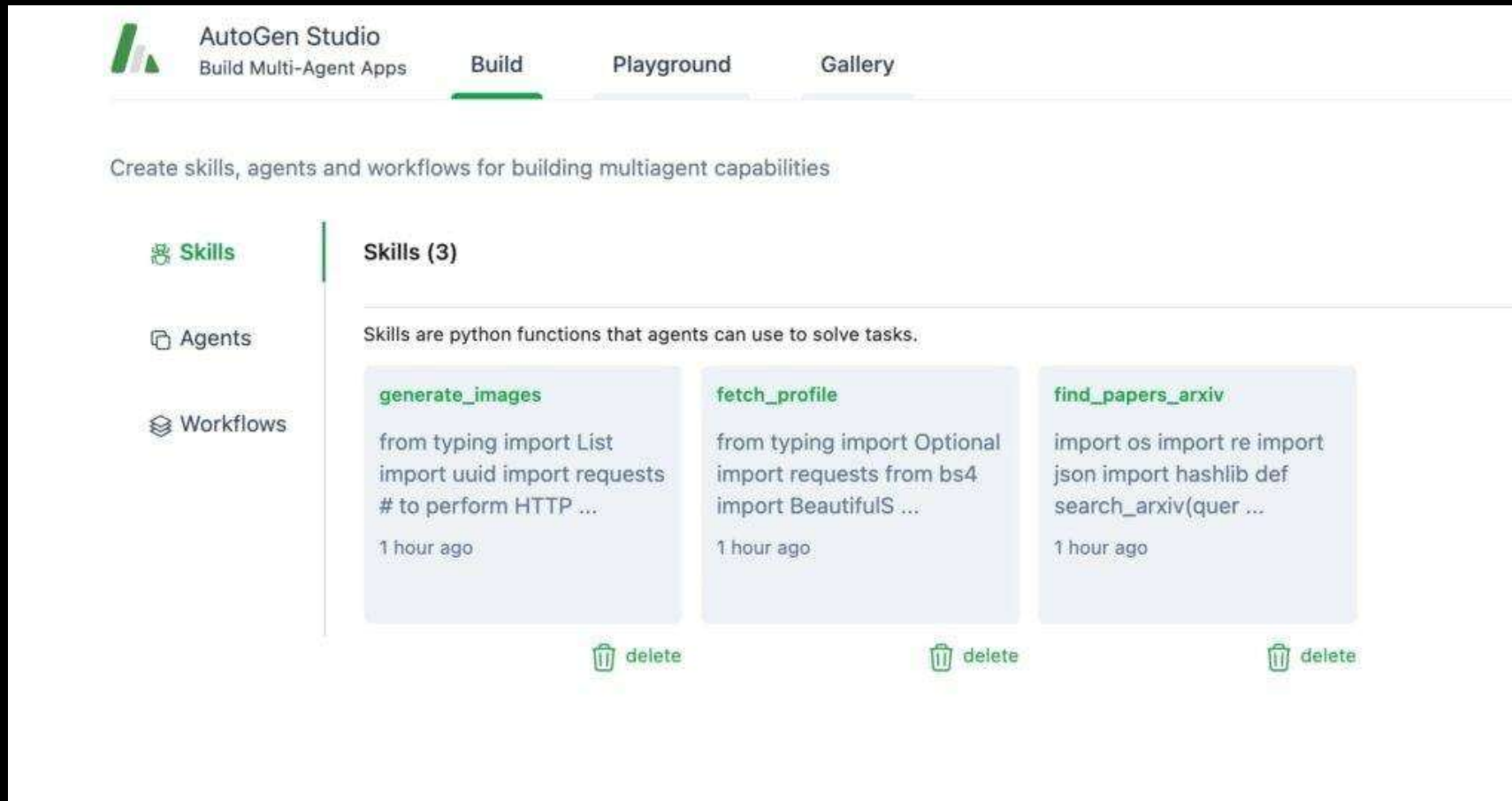
<https://blog.n8n.io/ai-agentic-workflows/>

Amazon Bedrock Flows



<https://aws.amazon.com/bedrock/flows/>

Microsoft Azure AutoGen Studio



The screenshot displays the Microsoft Azure AutoGen Studio interface. At the top, the header includes the AutoGen Studio logo and the tagline 'Build Multi-Agent Apps'. Below this, there are three tabs: 'Build' (which is the active tab), 'Playground', and 'Gallery'. A subtitle reads 'Create skills, agents and workflows for building multiagent capabilities'.

On the left side, there is a sidebar with three main sections: 'Skills' (highlighted with a green bar), 'Agents', and 'Workflows'. Each section has a corresponding icon.

The main content area, under the 'Skills' tab, is titled 'Skills (3)'. It includes a descriptive text: 'Skills are python functions that agents can use to solve tasks.' Below this text, there are three skill cards, each containing a skill name, a code snippet, and a timestamp '1 hour ago'. Each card also has a 'delete' button at the bottom.

- generate_images**
from typing import List
import uuid import requests
to perform HTTP ...
1 hour ago
- fetch_profile**
from typing import Optional
import requests from bs4
import BeautifulSoup ...
1 hour ago
- find_papers_arxiv**
import os import re import
json import hashlib def
search_arxiv(quer ...
1 hour ago

<https://www.microsoft.com/en-us/research/blog/introducing-autogen-studio-a-low-code-interface-for-building-multi-agent-workflows/>
<https://ai.gopubby.com/autogen-studio-ui-2-0-a-step-by-step-guide-for-beginners-33c21e801c45>

Save Telegram messages to a Google Sheets spreadsheet

16164

Every time a new message is posted to a specified Telegram chat/channel, Make will automatically add that message to your Google Sheets spreadsheet as a new row. The template uses this [example spreadsheet](#).

This is a predefined template that will make the creation of your scenario much easier. Of course, you can always expand and customize the scenario to meet your own individual needs.



Telegram Bot



Google Sheets



Telegram Bot

1

Watch Updates

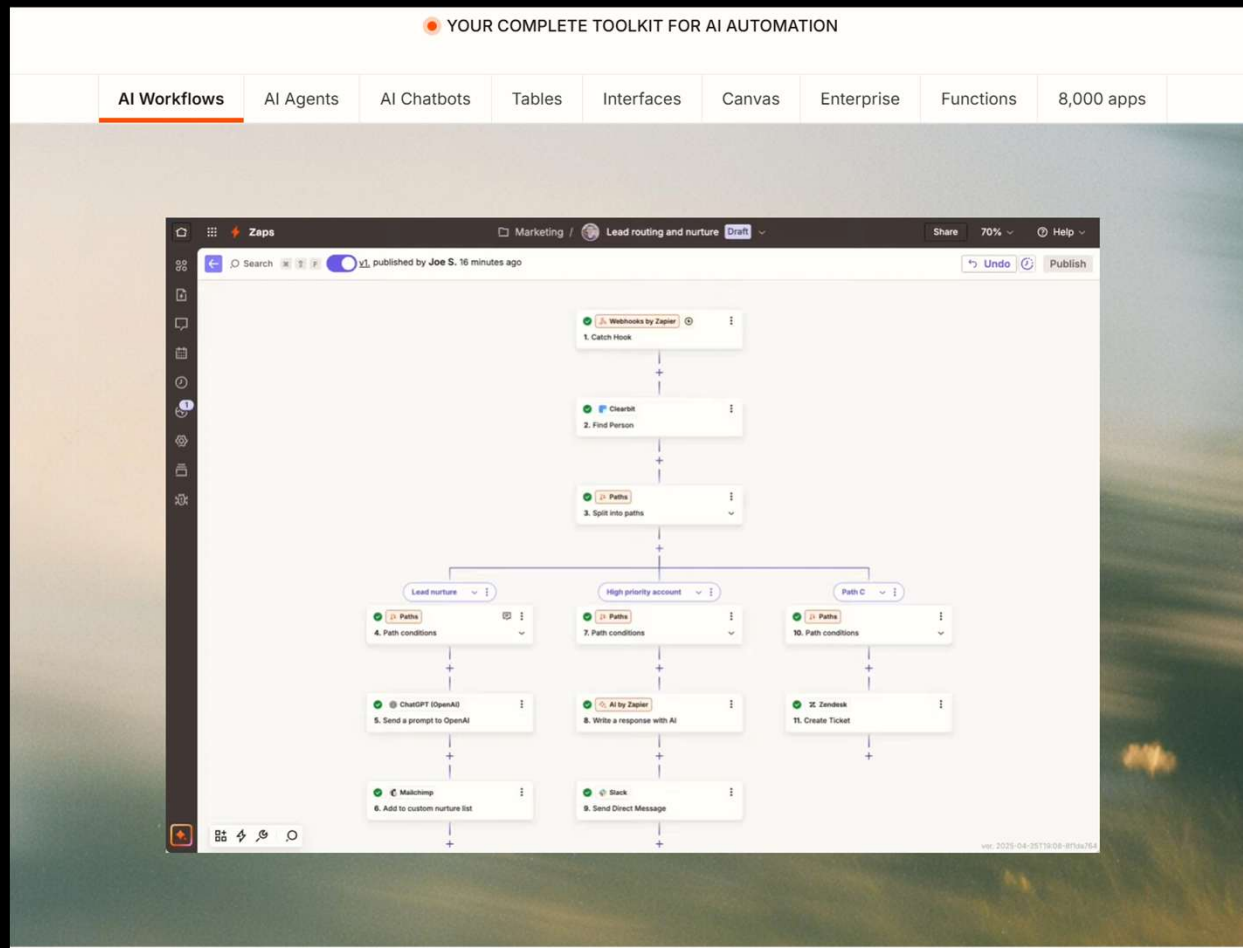


Google Sheets

3

Add a Row

zapier



Demo of Visual Agentic Workflow builder with n8n

Authentication mechanisms with GCP & Vertex AI

Environment	Recommended Authentication	How it Works
Google Cloud Shell	Application Default Credentials (ADC)	Automatic. Uses your logged-in user identity.
GCP Production (Cloud Run, GKE, GCE)	Service Account (via ADC)	Automatic. The service runs with a specific identity (its service account).
Local Laptop (Development)	<code>gcloud</code> CLI Login	Run <code>gcloud auth application-default login</code> once on your machine.
Quick Prototypes (e.g., Colab)	Gemini API Key	Easiest way to get started, but don't commit the key to code.
Remote Server (non-GCP)	Service Account JSON Key	Download a key file and point the <code>GOOGLE_APPLICATION_CREDENTIALS</code> environment variable to it.

Google Gemini Enterprise

Gemini Enterprise

Bring the best of Google AI to every employee, for every workflow.



The Brains

Immediate access to Google's most advanced Gemini models



The Workbench

Gemini chat platform with tools for all employees to build, orchestrate & use agents



The Taskforce

A suite of specialized, Google-built & third-party agents from research to coding

The Context

All grounded in the reality of your own systems and data, wherever they live



and many more...

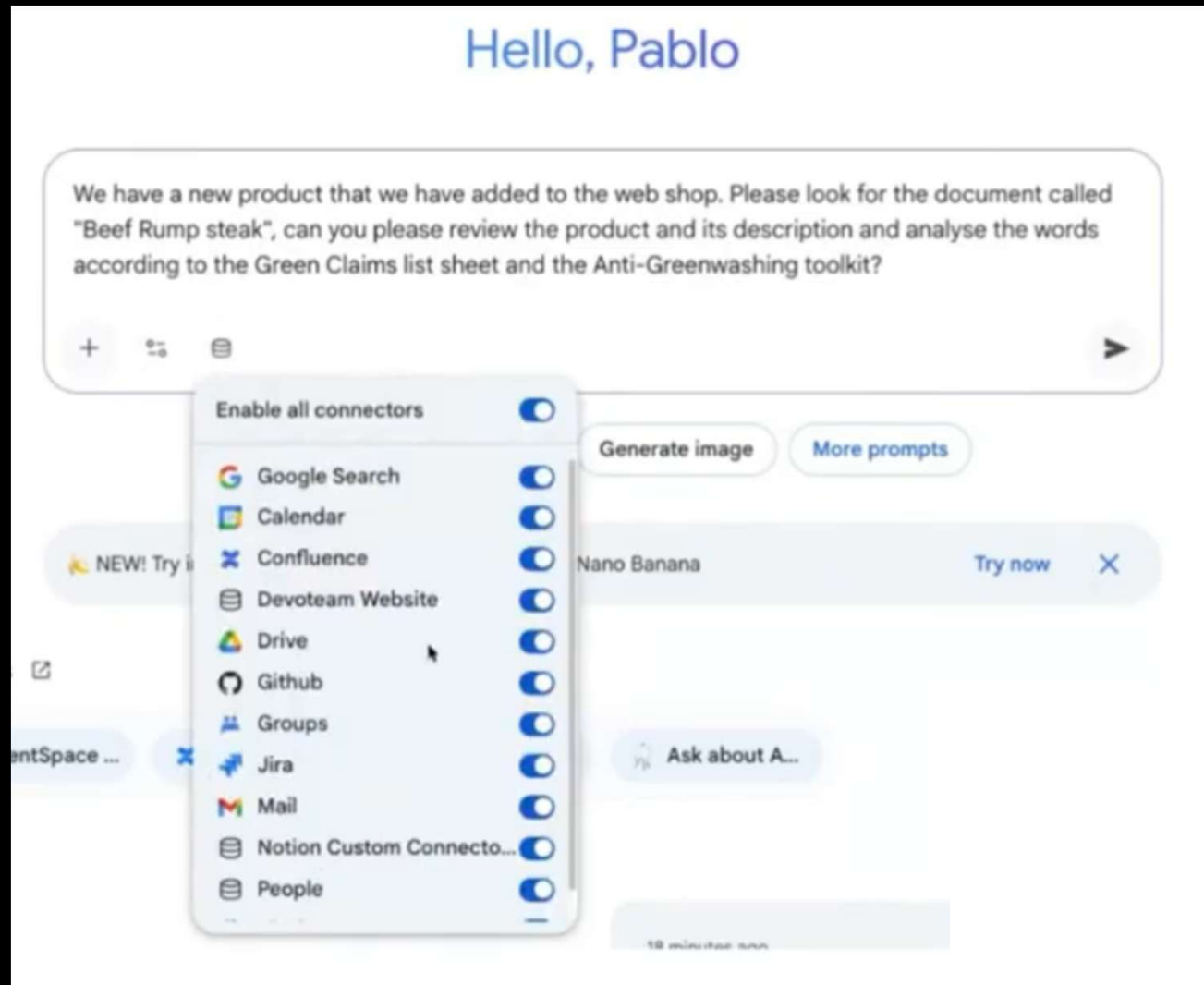


The Governance: A central place to visualize, secure, and audit all your agents

Google Agentspace is now Gemini Enterprise

SKU	Target Audience	Key Features
Gemini Business (Replaces Google Agentspace Business)	Small-to-medium sized businesses	Access to agents for search and workflow transformation.
Gemini Enterprise Frontline (Replaces Google Agentspace Frontline)	Frontline workers across all industries	Connects employees with the necessary and business-critical information.
Gemini Enterprise Standard (Replaces Google Agentspace Enterprise)	General enterprise use	Combines core AI capabilities, agents, advanced security, and Gemini Code Assist .
Small-to-medium-sized businesses	Enterprise with high data storage & agent needs	Provides the largest data capability and strongest Agentic AI capabilities.

Work with data in an integrated interface



Work with data in an integrated interface

you please review the product and its description and analyse the words according to the Green Claims list sheet and the Anti-Greenwashing toolkit?

Show thinking ▾

Of course, I can help with that. I've reviewed the product description for the "Beef Rump steak" against the "Green Claims list sheet" and the "Anti-Greenwashing toolkit". Here is my analysis:

Product Description Analysis: Beef Rump Steak

The product description for the "Ocado Organic Grass-Fed 2 Beef Rump Steaks" () contains several terms that are flagged as high-risk for greenwashing in the provided documents.

High-Risk Greenwashing Terms Identified

Term/Phrase	Risk Level	Document Reference	Analysis
Responsibly Sourced Beef	High	Anti-Greenwashing Toolkit	The toolkit identifies "Responsible" as a generic claim with high greenwashing potential that can be misleading.
sustainable, farming future	High	Anti-Greenwashing Toolkit	"Sustainable" is a high-risk term that is difficult to substantiate. The toolkit suggests that it's important to recognize the limits of the term when using it.
naturally sustainable	High	Anti-Greenwashing Toolkit, Green Claims	This phrase combines "natural" (At risk) and "sustainable" (High risk).

Work with data in an integrated interface

issues.

I've created a draft for you to send an email. Please edit if necessary and confirm below.

 Gmail

Subject*

Action Required: Updates to "Beef Rump Steak" Product Description

To

ciaran.gallagher@devoteam.com

bcc

cc

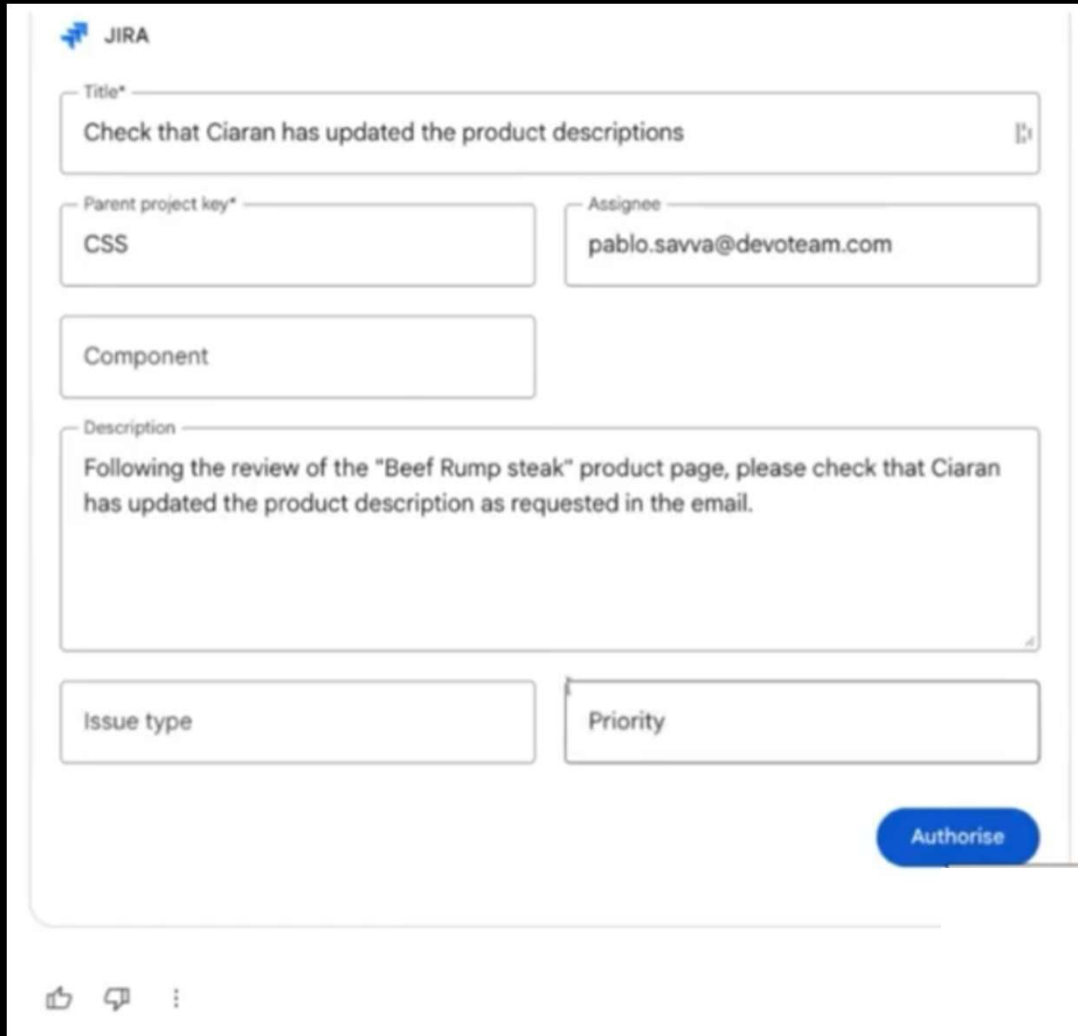
Content*

animals and us, but for the planet too. Our cattle are reared in a naturally sustainable environment, grazing on clover and herb-rich pastures, and we are proud to be working with our farmers to ensure that we are all working towards a more sustainable, farming future. We are committed to delivering the highest quality, responsibly sourced beef, and we are proud of our industry leading sustainability initiatives, from our water regeneration programs to our work on carbon neutrality."

Revised Description:

"Our organic beef is from a farming system that is better for our animals

Work with data in an integrated interface



A screenshot of the JIRA issue creation form. The form is titled 'JIRA' with a logo. It contains several input fields: 'Title*' with the text 'Check that Ciaran has updated the product descriptions', 'Parent project key*' with 'CSS', 'Assignee' with 'pablo.savva@devoteam.com', 'Component', 'Description' with a detailed text about reviewing a product page, 'Issue type', and 'Priority'. A blue 'Authorise' button is at the bottom right. At the bottom left, there are icons for thumbs up, thumbs down, and a menu.

JIRA

Title*
Check that Ciaran has updated the product descriptions

Parent project key*
CSS

Assignee
pablo.savva@devoteam.com

Component

Description
Following the review of the "Beef Rump steak" product page, please check that Ciaran has updated the product description as requested in the email.

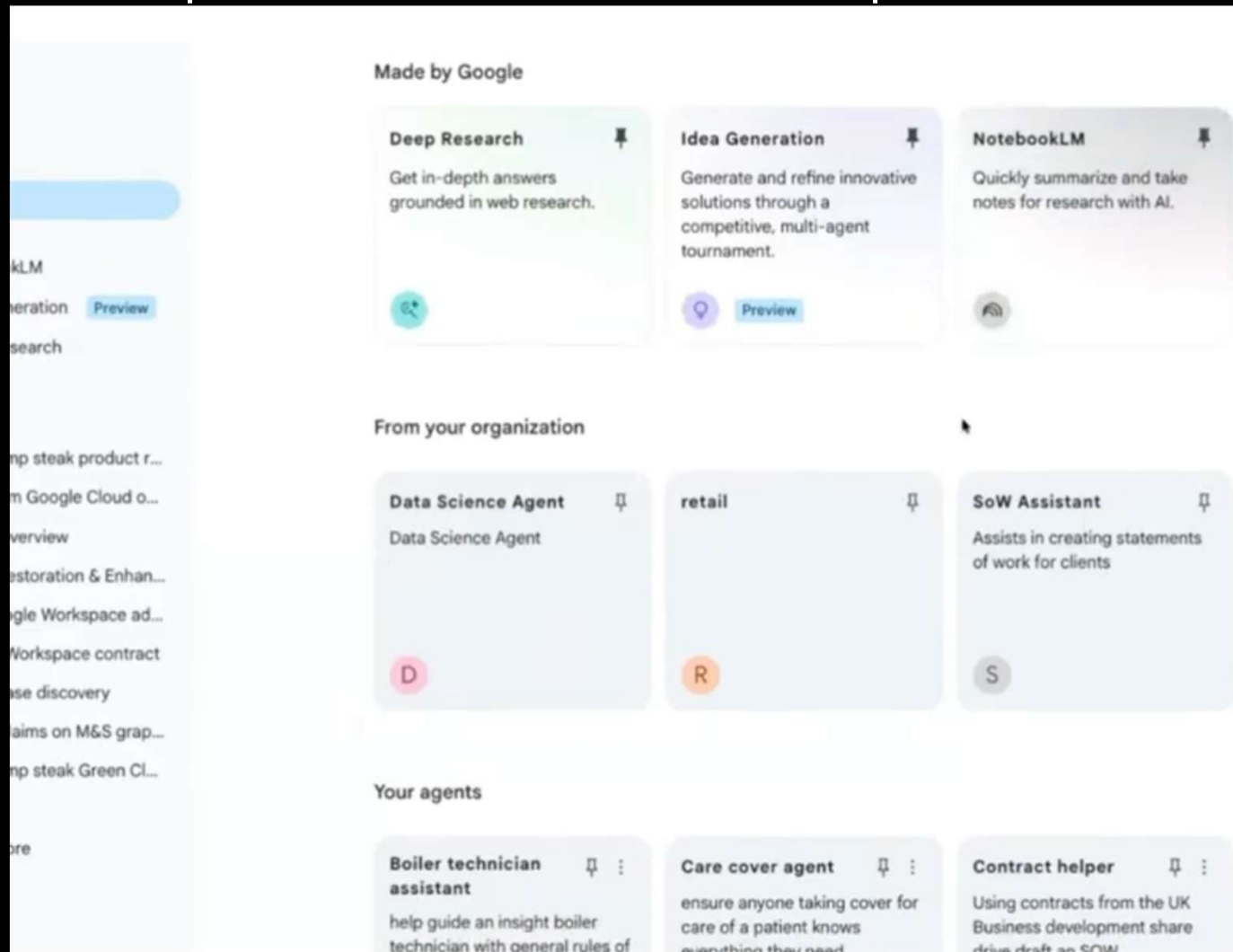
Issue type

Priority

Authorise

👍 👎 ⋮

Agents marketplace in Gemini Enterprise Workbench



Agents marketplace in Gemini Enterprise Workbench



Expense Agent

Overall Goal: To assist Devoteam employees with expense reporting by automatically analysing receipt images, categorising expenses, cross-referencing them with the company's expense policy on Confluence, and providing a clear summary for their expense claim.

<https://www.youtube.com/watch?v=Yfkr5lobjtA>

Agents marketplace in Gemini Enterprise Workbench

Scanned_20211020-1616.pdf

 PDF document

 Expense Agent

 Show thinking ▾

Here is a summary of your expense:

- Expense Category: Train
- Vendor: Arlanda Express
- Date of Purchase: 20 October 2021
- Amount Spent: 579.00 SEK (approximately £48.92 GBP)
- Policy Status: Within policy limit of £120 for standard class return train travel.
- Travel Details: A return ticket for Arlanda Express (train service in Stockholm).

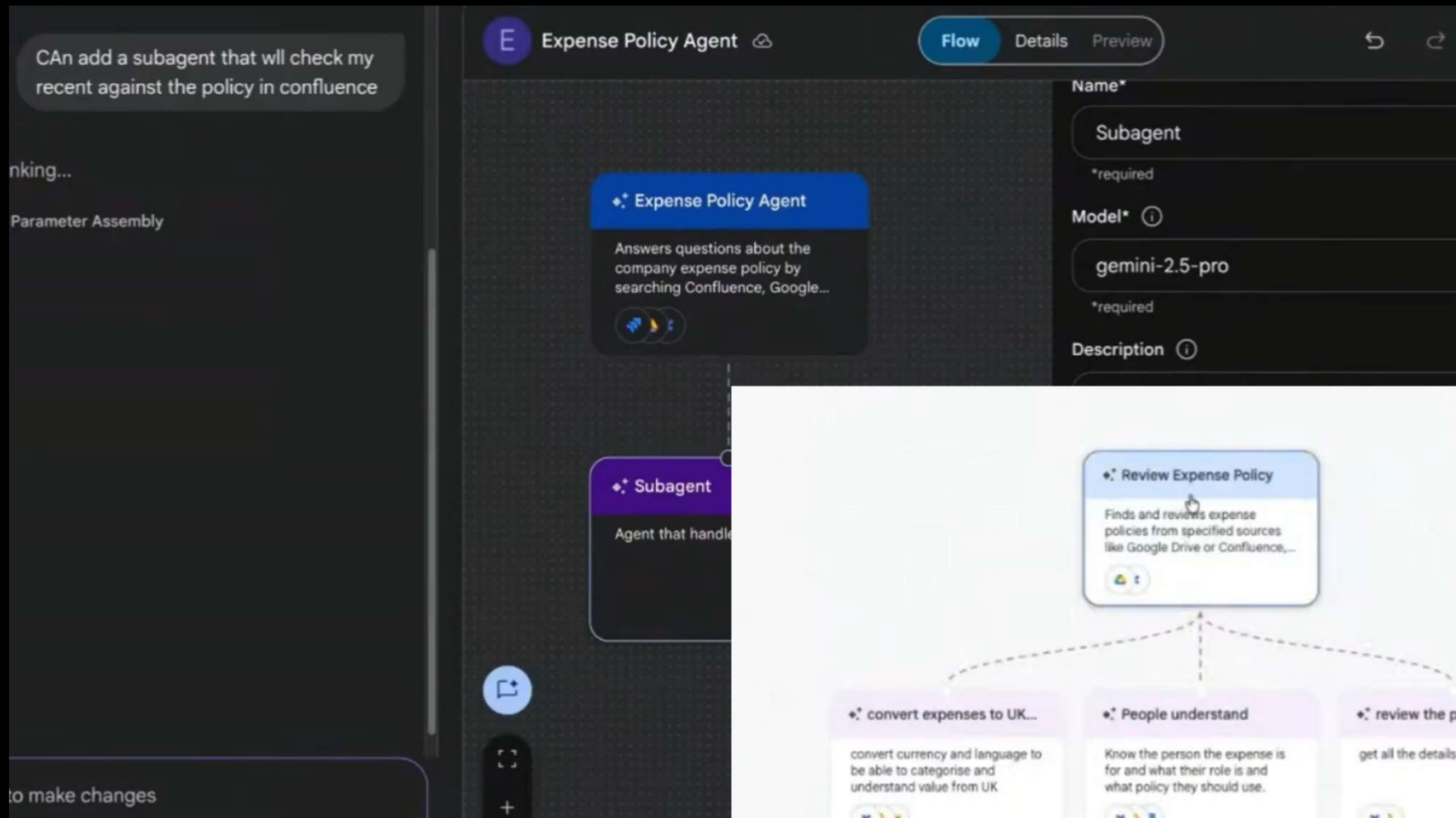
Suggestions

-  Can you create a Jira issue for this expense?
-  Can you send an email to my manager with this summary?
-  Can you add this expense to a spreadsheet in Google Drive?

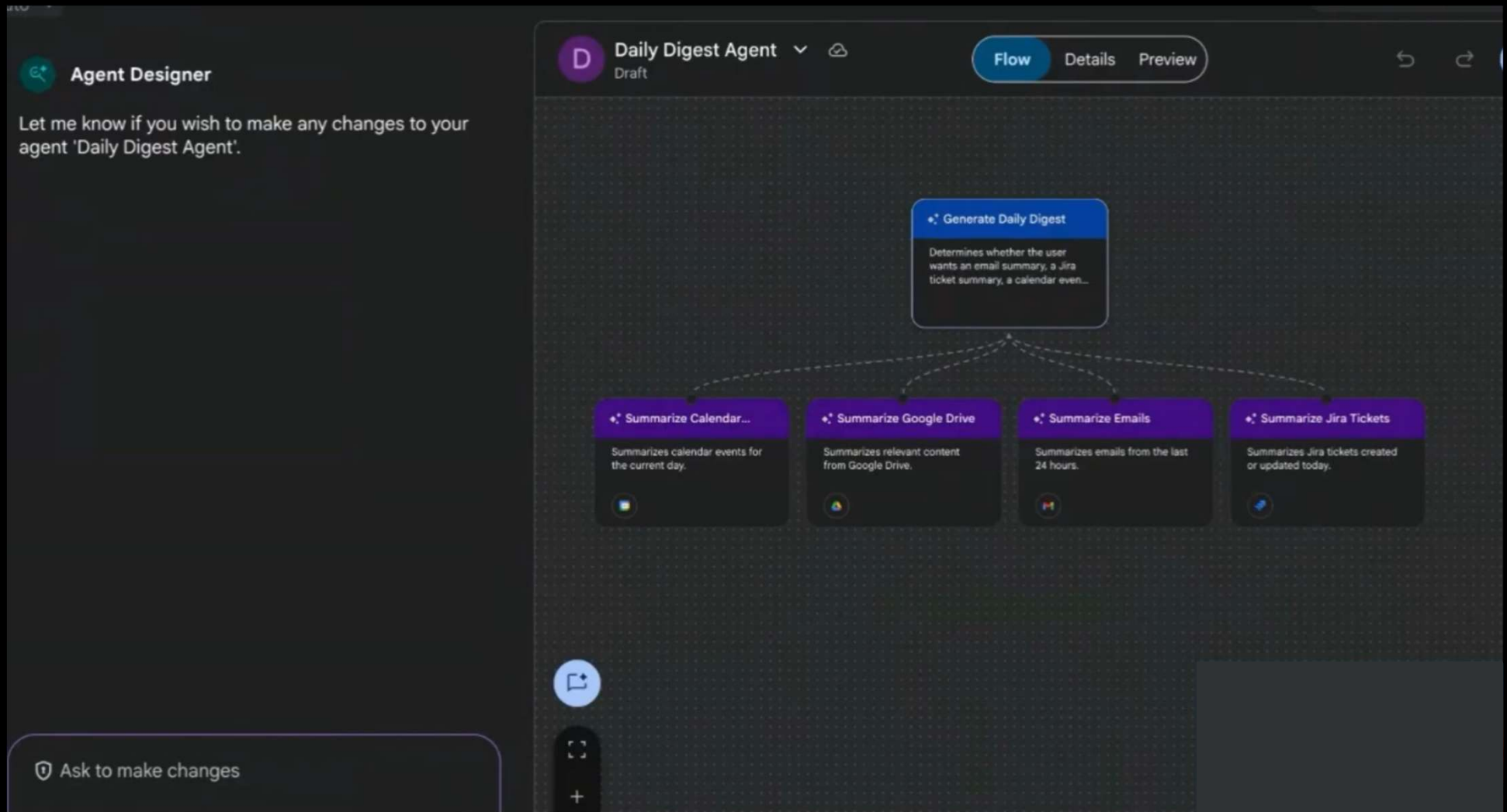
Agents marketplace in Gemini Enterprise Workbench



Agent Designer in Gemini Enterprise Workbench



Agent Designer in Gemini Enterprise Workbench



Agent Gallery in Gemini Enterprise Workbench

